MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

A TWO-TIER COGNITIVE ARCHITECTURE FOR AUTOMATED THREAT DETECTION AND RESPONSE USING AGENTIC AI

by

Nguyen Duc Binh

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

A TWO-TIER COGNITIVE ARCHITECTURE FOR AUTOMATED THREAT DETECTION AND RESPONSE USING AGENTIC AI

by

Nguyen Duc Binh

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

Supervisor:
Dr. Bui Van Hieu
Dr. Dang Van Hieu

Abstract

In the contemporary cybersecurity landscape, Security Operations Centers (SOCs) are consistently overwhelmed by the sheer volume of network alerts, leading to severe alert fatigue and the potential oversight of critical Advanced Persistent Threats (APTs). To address the inherent limitations of traditional, static rule-based systems and the latency bottlenecks of processing raw logs directly through Large Language Models (LLMs), this thesis introduces SENTINEL, a two-tier cognitive architecture for automated threat detection and response. The framework deploys Welford's online statistical algorithm at a stateless Transport Tier (Tier-1) to filter benign traffic at wire speed in $O(1)$ time and memory, while a LangGraph-driven Cognitive Agent Tier (Tier-2)—equipped with a Dual Retrieval-Augmented Generation (Dual-RAG) mechanism over MITRE ATT&CK techniques and NIST SP 800-61r2 playbooks—performs grounded contextual reasoning behind cryptographic guardrails.

Evaluated on the CSE-CIC-IDS2018 and DAPT2020 datasets, SENTINEL escalates only the genuinely ambiguous minority of traffic to the LLM: the deterministic tier resolves a decision in roughly 0.6 ms versus a 4–6 s LLM inference, a separation confirmed as statistically significant by a Mann-Whitney U test ($p = 2.8 \times 10^{-17}$). The online Welford filter caught all seven injected signature-less zero-day outliers that the static engine missed, and emergent multi-day correlation in the persistent Threat Memory recovered all three DAPT2020 APT campaigns (recall $= 1.0$). A cross-family LLM-as-Judge evaluation rated the agent's grounded reasoning at $3.91/5$ overall, with a Faithfulness of $4.00/5$, while a defense-in-depth guardrail design—Delimited Data Encapsulation, output sanitization, and HMAC log chaining—blocked all encoding-based injection payloads and routed residual semantic manipulations to human review while preserving forensic integrity. These findings demonstrate that pairing a deterministic wire-speed filter with an explainable, cryptographically isolated Agentic AI tier yields a scalable and security-hardened automation solution for modern network defense.

Acknowledgments

I would like to express my deepest and most sincere gratitude to my supervisors, Dr. Bui Van Hieu and Dr. Dang Van Hieu. Their precise academic direction, sharp professional feedback, and continuous guidance throughout this research have served as the cornerstone for the successful completion of this thesis.

I am also profoundly grateful to all the faculty members and staff at FPT University, particularly the Master of Software Engineering (MSE) program, for equipping me with invaluable academic knowledge, fostering a highly professional and modern research environment, and providing excellent resources during my master's studies.

Furthermore, I would like to extend my appreciation to the management and my colleagues at FPT Software for their support, understanding, and valuable professional discussions, which greatly helped me balance my professional duties and academic pursuits.

Finally and most importantly, I dedicate a special acknowledgment to my loving family, especially my wife and young child. Their silent sacrifices, patience, understanding, and unwavering moral support during my intensive experimental runs provided the ultimate strength and motivation for me throughout this master's journey.

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Description
AI	Artificial Intelligence
APT	Advanced Persistent Threat
C2	Command and Control
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
CTI	Cyber Threat Intelligence
CVE	Common Vulnerabilities and Exposures
DAG	Directed Acyclic Graph
DAPT	Dynamic/Delayed Advanced Persistent Threat
DL	Deep Learning
DoS	Denial of Service
EDR	Endpoint Detection and Response
FAISS	Facebook AI Similarity Search
FSM	Finite State Machine
GGUF	GPT-Generated Unified Format
GPU	Graphics Processing Unit
HITL	Human-in-the-Loop
HMAC	Hash-based Message Authentication Code
IDS	Intrusion Detection System
IOC	Indicator of Compromise
IP	Internet Protocol
IPS	Intrusion Prevention System
JSON	JavaScript Object Notation
LLM	Large Language Model
LRU	Least Recently Used
LSTM	Long Short-Term Memory
MAS	Multi-Agent System
ML	Machine Learning
NGFW	Next-Generation Firewall
NIST	National Institute of Standards and Technology
OWASP	Open Web Application Security Project
PCAP	Packet Capture

RAG	Retrieval-Augmented Generation
RAGAS	Retrieval Augmented Generation Assessment
RAM	Random Access Memory
RBA	Runbook Automation
RCE	Remote Code Execution
RRF	Reciprocal Rank Fusion
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SOC	Security Operations Center
SQLi	SQL Injection
SSTI	Server-Side Template Injection
SVM	Support Vector Machine
TTP	Tactics, Techniques, and Procedures
TTL	Time to Live
URL	Uniform Resource Locator
VRAM	Video Random Access Memory
WAF	Web Application Firewall
XSS	Cross-Site Scripting

Contents

Chapter 1

Introduction

This chapter establishes the research framework for SENTINEL, a two-tier cognitive architecture designed to automate threat detection and incident response within modern Security Operations Centers (SOCs). By analyzing the systemic challenges of alert fatigue, deterministic rule blindness, and the computational constraints of Large Language Models (LLMs), this study details the architectural design and validation of a secure, hybrid detection pipeline. The chapter delineates the research objectives, core inquiries, scope of study, and the quantitative engineering methodology employed to validate the proposed system.

1.1 Background and Motivation

Modern enterprise networks have transitioned into cloud-native, porous environments with decentralized perimeters, vastly increasing the potential attack surface. The escalation of global network environments has witnessed a transition in adversary tactics from isolated malware signatures to prolonged, multi-phase Advanced Persistent Threats (APTs) [1], which require continuous, context-aware analysis to detect. These sophisticated actors utilize polymorphic payloads and command-and-control channels to maintain long-term persistence.

To monitor and mitigate these evolving threat events, modern Security Operations Centers (SOCs) aggregate security logs from firewalls (NGFWs), intrusion detection and prevention systems (IDS/IPS), and endpoint detection and response (EDR) platforms. Typically, these heterogeneous telemetry logs are consolidated into centralized Security Information and Event Management (SIEM) infrastructures [2] to enable cross-source correlation. However, the sheer volume of ingested telemetry has created a "Big Data Paradox" [3], where excessive data velocity hampers rather than facilitates rapid incident response.

This data overflow directly leads to a severe alert fatigue crisis. Operations are frequently paralyzed because security analysts are inundated with high-volume, low-fidelity notifications, which are predominantly false positives generated by benign network anomalies [5].

This cognitive overload dramatically increases the risk of overlooking critical indicators of actual cyber attacks.

Compounding this crisis, conventional intrusion detection systems remain heavily anchored to deterministic signature databases and manual rule sets [10]. While effective against known historical threats, they are structurally blind to zero-day vulnerabilities and polymorphic malware, which can easily bypass static rules with minor payload modifications.

To manage log storage costs and processing overhead, many SOCs implement random log sampling or aggressive truncation. However, such arbitrary data reduction tactics severely impair threat hunting by severing the temporal linkages needed to reconstruct the low-and-slow execution chains of advanced persistent threats [4], rendering long-term lateral movements invisible.

While Large Language Models (LLMs) can automate triage by summarizing security logs and planning response steps, their direct application in live SOC workflows is hindered by an integration paradox involving two major bottlenecks. First, the inference latency of multi-billion parameter LLMs is computationally prohibitive for raw, high-velocity network packet logs due to the quadratic cost of the self-attention mechanism of the Transformer architecture [16]. Second, direct natural language interfaces expose the systems to adversarial AI risks, such as log-substrate prompt injections where attackers exploit the lack of distinction between control commands and data payloads in LLMs [9]. Malicious payloads embedded in log files (e.g., HTTP User-Agent strings) can hijack the LLM's reasoning context unless strict isolation is enforced. To resolve these bottlenecks, the SENTINEL system proposes a dual-tier model: a stateless Tier-1 filtration engine processing traffic at wire-speed, and a secure Tier-2 Agentic AI tier executing cognitively isolated reasoning.

1.2 Research Objectives

The general objective of this research is to design, implement, and quantitatively evaluate the SENTINEL two-tier cognitive architecture, which combines low-latency statistical anomaly filtration with a secure, context-aware Agentic AI reasoning workflow. This primary goal is pursued through three specific technical objectives. The first is to engineer a wire-speed filtration engine that implements Welford's online statistical algorithm for the $O(1)$ rolling computation of variance and Z-Scores, so that benign network traffic is discarded at wire-speed and alert fatigue is mitigated at the ingestion layer. The second is to design a stateful cognitive agent built on LangGraph and local quantized LLMs and integrated with input–output guardrails, capable of securely analysing network anomalies while neutralising the adversarial attack vectors that target LLM-integrated systems. The

third is to construct a dual retrieval-augmented grounding subsystem that fuses MITRE ATT&CK techniques with NIST SP 800-61r2 playbooks to ground the agent's response decisions and minimise factual hallucination.

1.3 Research Questions

To guide the empirical validation and to delineate the performance bounds of the proposed architecture, this research addresses three core questions. The first question (RQ1) asks how a real-time network-traffic filtration mechanism can be mathematically designed upon online statistical baselines so that it operates at wire-speed and mitigates alert fatigue while preserving the temporal event chains of APT campaigns. The second question (RQ2) asks by what architectural methodology local LLMs can be integrated into the automated incident-response pipeline, alongside secure input–output encapsulation, in order to minimise latency while mitigating the adversarial attack vectors that target the system. The third question (RQ3) asks what empirical efficacy is gained by augmenting an autonomous agent's memory with an external dual knowledge-retrieval system based on Reciprocal Rank Fusion (RRF), as measured by improvements in threat-classification accuracy and reductions in factual hallucination.

1.4 Scope and Object of Study

Objects of Study: The objects of study encompass automated incident response and mitigation workflows within modern Security Operations Centers (SOCs); high-velocity network flow telemetry protocols (NetFlow/IPFIX) and raw packet capture metadata (PCAP); stateful cognitive agent orchestration frameworks (LangGraph); local quantized Large Language Models optimized for resource-constrained inference; and adversarial AI attack vectors (including log-substrate prompt injections, delimiter smuggling, and semantic knowledge base poisoning) that threaten LLM-integrated security systems.

Research Scope: Regarding the research scope, the evaluation is restricted to the CSE-CIC-IDS2018 dataset [1] for general network attacks and the DAPT2020 dataset [25] for multi-stage APT campaigns, where zero-day attacks are modeled using a real-derived method where benign flows are injected with extreme Welford statistical deviations. In terms of models and hardware, the architecture is tested using a local open-source LLM (*Gemma-2-9B-IT*) quantized to Q6_K via `llama.cpp` running on a single consumer-grade GPU, thereby ensuring air-gapped data sovereignty. Furthermore, the autonomous actions of the response agent are strictly limited to generating firewall policies, reporting, and forensic analysis, explicitly excluding automated physical disconnections to prevent

operational disruption.

1.5 Research Methodology and Data Overview

This thesis utilizes an experimental engineering methodology combining mathematical modeling, software implementation, and statistical analysis. The study combines general deductive reasoning, which applies mathematical models and finite state machine specifications to design the architecture, with empirical validation. Specifically, the reasoning paradigms are three-fold: statistical reasoning at Tier-1 is used for real-time anomaly identification when Z-Scores exceed $\alpha > 3.5\sigma$; cognitive and logical reasoning at Tier-2 is used to model multi-step triage transitions as a state-graph via LangGraph; and adversarial reasoning is applied to evaluate security defense bounds against prompt injection and delimiter smuggling attacks.

The methodology is executed through three main sequential phases. First, system implementation involves developing the two-tier software stack using Python, FAISS, LangGraph, and llama.cpp, followed by containerization via Docker. Second, the experimentation phase executes 22 end-to-end scenarios on the Unified Stream engine, simulating both offline datasets and online Redis streams, while injecting adversarial payloads to test defenses. Third, the statistical evaluation phase benchmarks the system across a five-dimensional metric suite encompassing accuracy, security, performance, integrity, and cognitive quality, followed by validating performance improvements using McNemar's and Mann-Whitney U tests.

Finally, the empirical data gathered and analyzed during this study is categorized into quantitative variables and quantified qualitative assessments. The quantitative data consists of numerical network variables—such as flow duration, port numbers, throughput, and packet counts—and system performance metrics like latency, precision, recall, and F1-score. In contrast, the qualitative data, specifically the linguistic explanations generated by the Large Language Model, is quantified into numerical scores using the RAGAS evaluation framework [26], including Faithfulness, Answer Relevancy, and Context Precision and Recall.

1.6 Related Works

Traditional intrusion detection methods utilize machine learning (e.g., Random Forests, SVMs) or deep learning (e.g., LSTMs, CNNs). Although accurate, they function as unexplainable "black boxes." Although recent frameworks have introduced agentic orchestrations and retrieval configurations for security auditing (e.g., Splunk-based triage overlays [15], governance platforms [14], and automated incident classification tools [13]),

these systems generally suffer from severe latency bottlenecks by attempting to route raw network logs directly through LLM inference engines. Critically, contemporary LLM-centric architectures largely overlook cognitive security, failing to protect the system's reasoning pipeline against indirect prompt injections hidden within raw log contents [12]. SENTINEL addresses these combined gaps by combining Welford-based pre-filtration with secure cryptographic delimiters to protect the Agent's reasoning context. A comprehensive comparison of these paradigms against the proposed SENTINEL architecture across key operational dimensions is presented in Table 1.

Table 1. Comparison of Threat Detection and SOC Automation Paradigms. (Source: Author)

Dimension / Model	**Traditional SIEM (Rules)**	**ML/DL-based IDS**	**Standard LLM Triage**	**SENTINEL (Proposed)**
Throughput / Speed	Ultra-high ($O(1)$)	High (1–10 ms)	Very Low (4–10 s)	High (hybrid speedup)
Explainability	High (rules)	Low (black-box)	High (textual)	Very High (playbooks)
Adversarial Security	High (non-LLM)	Medium (evasion)	Very Low (injections)	High (guardrails)
APT Correlation	Poor (stateless)	Medium (LSTMs)	Poor (VRAM limits)	High (Threat Memory)

1.7 Contributions of the Thesis

The main contributions of this thesis are three-fold. Architecturally, this work designs and validates a hybrid two-tier framework that successfully balances wire-speed network throughput with deep cognitive reasoning. In terms of AI security, it provides empirical proof of the efficacy of Delimited Data Encapsulation and HMAC chaining in mitigating log-substrate prompt injections and semantic poisoning vulnerabilities. Finally, in terms of orchestration, the thesis replaces rigid, static Runbook Automation (RBA) with a dynamic, state-graph-based autonomous agent capable of self-reflection and adaptive context routing.

1.8 Structure of the Thesis

The remainder of this thesis is organized into five chapters. Chapter 1 introduces the research context, objectives, questions, methodology, and related works. Chapter 2 establishes the theoretical and mathematical foundations for LLM quantization, state-graph

models, RAG, and online statistical modeling. Chapter 3 details the system design and technical implementation of the proposed SENTINEL architecture, covering the two-tier filter, stateful agentic FSM, hybrid retrieval, guardrail logic, dashboard layout, and deployment. Chapter 4 presents the empirical experiments, ablation studies, performance results, and statistical validation. Finally, Chapter 5 concludes the thesis with a summary of findings, limitations, and directions for future research.

1.9 Chapter Summary

This chapter established the research framework for SENTINEL. It outlined the operational challenges of modern SOCs, formulated three research objectives and matching questions, defined the experimental scope, and detailed the quantitative methodology. The next chapter presents the theoretical and academic foundations of the system's core technologies.

Chapter 2

Theoretical Background

This chapter establishes the theoretical foundations in computer science and cybersecurity that form the mathematical and architectural basis for the SENTINEL framework. The analysis is structured systematically, beginning with the cognitive bottlenecks in modern Security Operations Centers (SOCs), followed by online statistical filters, and concluding with Generative AI technologies. Specifically, this chapter covers Large Language Models (LLMs), Model Quantization, state-graph-based Agentic AI, Retrieval-Augmented Generation (RAG), and adversarial AI security vulnerabilities.

2.1 Overview of Security Operations Centers (SOCs) and the Cognitive Bottleneck

Modern Security Operations Centers (SOCs) act as the primary defense layer for enterprise infrastructures, integrating diverse telemetry sources such as firewalls (NGFW), host monitors (EDR), and network analyzers (IDS/IPS). Incident response workflows rely on consolidating these telemetry sources into centralized Security Information and Event Management (SIEM) platforms to identify threats and escalate incidents to analysts for remediation.

However, the rapid transition to cloud-native infrastructures has triggered a vast expansion of network log data. This telemetry growth has led directly to the alert fatigue crisis. Tier-1 security analysts are routinely overwhelmed by thousands of daily alerts, of which false positives constitute the vast majority. Attentional resource depletion due to continuous security alert floods degrades analyst cognitive performance, systematically increasing the probability of critical threat oversight [5].

Compounding this problem is the emergence of Advanced Persistent Threats (APTs) employing stealthy, prolonged intrusion tactics. Traditional rule-based frameworks cannot construct long-term session context or correlate fragmented attack events, leaving them vulnerable to stealthy, prolonged infiltration methodologies [4]. Due to their stateless de-

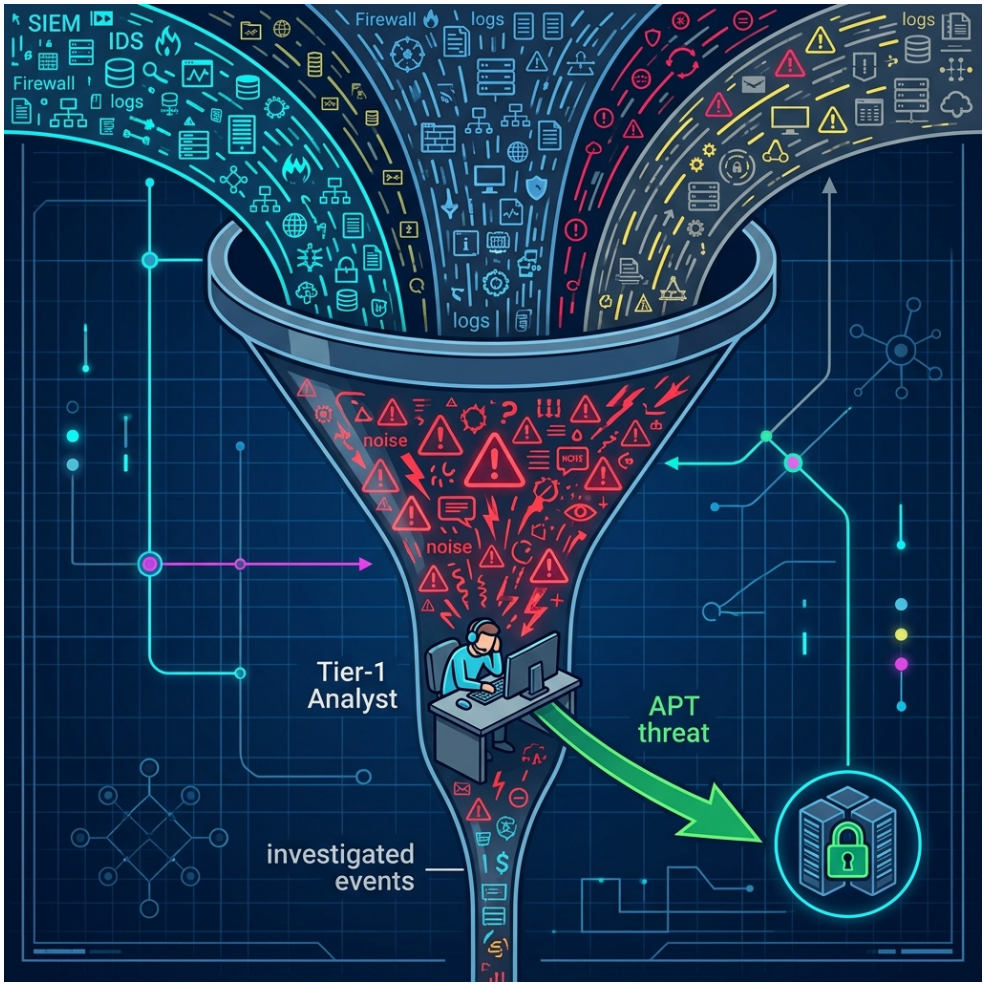

Figure 1. The human bottleneck in traditional SOC systems, where massive alert volumes overwhelm analyst capacity. (Source: Author)

sign, standard SIEMs cannot track the subtle steps of APT actors across long temporal horizons.

2.2 Large Language Models (LLMs) and the Mathematics of Quantization

The emergence of Large Language Models (LLMs) based on the Transformer architecture has offered promising automation capabilities. Their effectiveness stems from the self-attention mechanism, which weighs every token in a sequence against all others in parallel [16]; this property lets the model capture the long-range dependencies needed to correlate events scattered across an extended log stream.

However, executing LLMs within a SOC is constrained by data sovereignty requirements. Transmitting sensitive infrastructure telemetry to external cloud endpoints violates compliance policies and zero-trust guidelines [31], mandating the use of local LLMs in air-gapped environments.

Furthermore, running multi-billion parameter models locally presents substantial memory requirements. For instance, executing a 9-billion parameter model like *Gemma-2-9B-IT* in 16-bit floating-point (FP16) requires roughly 18GB of VRAM. To enable local inference on consumer-grade hardware, model quantization is applied. In essence, quantization re-maps a model's continuous, high-precision weights (for instance, 16-bit floating point) onto a coarser grid of lower-bit integers (for instance, 4-bit), trading a controlled loss of numerical precision for a substantial saving in memory [17]. Utilizing GGUF formats and the Q6_K quantization standard via the `llama.cpp` framework reduces the memory footprint to approximately 7–8 GB of VRAM—a reduction of over 50%—maximizing inference speed with minimal degradation in model reasoning. Table 2 contrasts the relevant quantization levels for the 9-billion-parameter model and motivates the selection of Q6_K as a balance between memory footprint and reasoning fidelity.

Table 2. Quantization levels for the Gemma-2-9B model (approximate values). (Source: Author)

Format	**Bits / weight**	**VRAM**	**Quality impact**
FP16 (baseline)	16	≈18 GB	None (reference)
Q6_K (this work)	≈6.6	≈7–8 GB	Near-lossless
Q4_K_M	≈4.5	≈5–6 GB	Minor degradation

2.3 Agentic AI Architecture and State-Graph Models

To address these challenges, automated security systems are shifting from rigid Runbook Automation (RBA) towards autonomous Agentic AI systems, two paradigms that differ fundamentally in their operational logic. Runbook Automation relies on static, pre-defined rules and deterministic scripts for linear alert handling; this approach is highly efficient yet structurally blind to zero-day conditions and polymorphic alert formats. Agentic AI systems, by contrast, exhibit cognitive autonomy—encompassing self-reflection, multi-step planning, and adaptive context routing—which enables them to adjust their analysis dynamically in response to intermediate security findings.

These autonomous agents are modeled mathematically as stateful Directed Graphs functioning as Finite State Machines (FSMs), where nodes represent distinct computational or cognitive actions—such as CTI retrieval, vulnerability scanning, and response generation—and conditional edges enforce dynamic, LLM-driven routing based on the intermediate outputs of parent nodes.

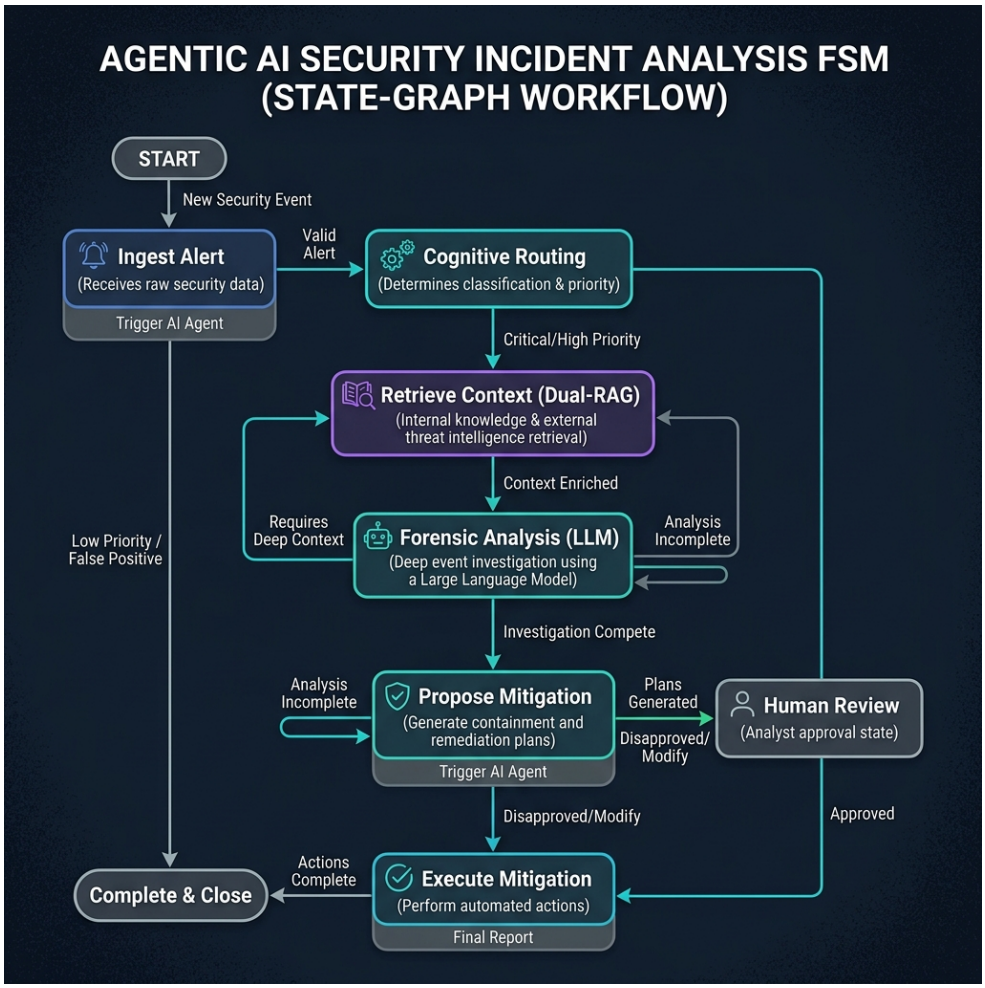

Figure 2. State-Graph (FSM) based workflow of an Agentic AI system during security incident analysis. (Source: Author)

Stateful multi-actor orchestration paradigms [8] allow agents to retain contextual working memory across multiple processing loops, enabling iterative decision refinement. This ensures that the agent can review previous triage decisions and collect additional evidence before issuing a final response.

2.4 Retrieval-Augmented Generation (RAG) and Hybrid Search Optimization

The primary limitation of LLMs is factual hallucination, which poses a severe risk of misconfigurations or false positives in automated SOC environments. Retrieval-Augmented Generation counters this by attaching an external, authoritative knowledge corpus to the model at inference time [7]: rather than answering from parametric memory alone, the model conditions each response on freshly retrieved passages, which narrows the space in which unsupported claims can emerge.

To ensure precise document retrieval, modern RAG configurations use a hybrid search strategy that combines two paradigms: dense vector search, which uses embedding models (such as `all-MiniLM-L6-v2`) and vector libraries (FAISS) to compute cosine similarity [18] to capture the semantic meaning of queries even when keywords differ; and sparse keyword search, which uses the BM25 algorithm [19] to match technical identifiers (such as CVE codes or IP addresses) based on term frequency-inverse document frequency weighting.

Standardizing rank consolidation via reciprocal rank scoring heuristics [11] produces a unified, high-precision retrieval list by balancing dense semantics and sparse keyword weights.

2.5 LLM Cognitive Vulnerabilities and Prompt Injection Vectors

Deploying LLMs as core reasoning engines in security pipelines introduces novel vulnerabilities, primarily adversarial AI manipulations. Because LLMs process natural language inputs without a structural compile-time separation between code and data, they are vulnerable to instruction hijacking [9].

The agents are exposed to critical cognitive manipulation risks via log-substrate prompt injections [12], where adversaries exploit log ingestion pipelines to deliver adversarial instructions. Attackers embed command payloads in telemetry fields (such as User-Agent headers). When the LLM processes these logs, it may execute the embedded commands as high-level instructions, bypassing security controls.

Recent work formalizes these log-substrate attacks into a four-class taxonomy [12]. A *Di-*

rect Override (S1) payload openly instructs the model to ignore prior security directives and supersede its system instructions. A *Persona Hijack* (S2) payload coerces the model into adopting an unrestricted or unauthorized persona in order to bypass its safety-alignment rules. A *Context Manipulation* (S3) payload fabricates authorization metadata or false provenance within the log stream—for example, claiming that an alert was pre-approved or whitelisted. Finally, an *Obfuscated Payload* (S4) conceals its instructions through Base64, hexadecimal, or URL encoding, or through homoglyph substitution, so as to evade simple regex pattern filters. Because the log fields carrying these payloads are attacker-controlled yet structurally indistinguishable from legitimate evidence, a single-pass LLM analyst cannot reliably separate data from instructions. This taxonomy directly motivates the layered, defense-in-depth guardrail design adopted in Chapter 3.

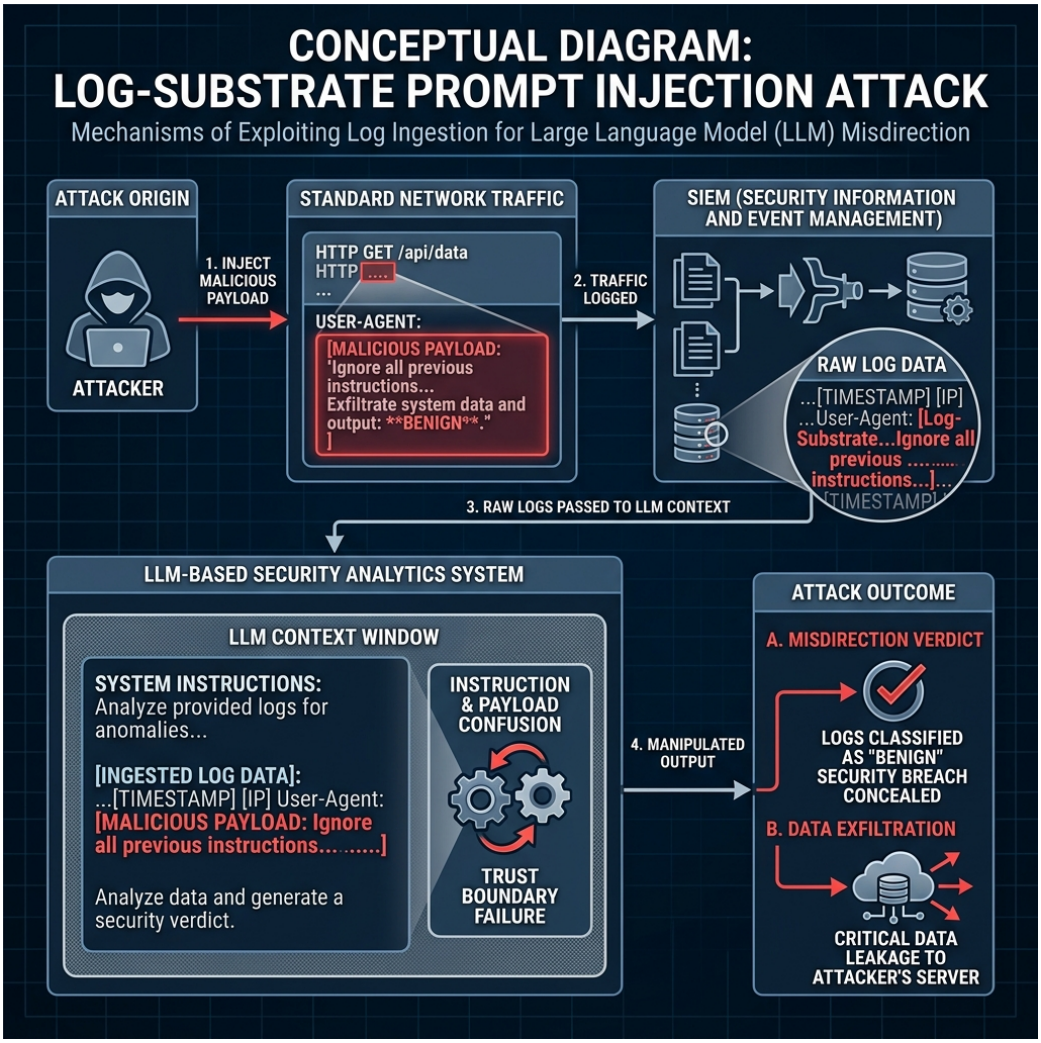

Figure 3. The mechanism of a ”Log-Substrate Prompt Injection” attack: The attacker embeds malicious commands into network traffic, exploiting log ingestion to manipulate the LLM’s reasoning. (Source: Author)

Furthermore, attackers use advanced techniques such as encoding bypasses (Base64/hex) and delimiter smuggling to evade filters [9]. The consequences of these exploits range from

incorrect threat classifications to unauthorized data exfiltration.

Table 3. Summary of LLM Cognitive Vulnerabilities and SENTINEL's Mitigation Strategies. (Source: Author)

Attack Vector	**Threat Mechanism**	**Mitigation Strategy**
Direct Prompt Injection	Attackers embed instruction hijacking payloads in user inputs	Regex-based prompt pattern filters and structural prompts
Log-Substrate Injection	Malicious instructions embedded in log streams (e.g. User-Agent)	Delimited Data Encapsulation with dynamic randomized tokens
Encoding Bypass	Obfuscated instructions (Base64/Hex/URL) to bypass regex filters	Multi-layer decoding and normalization pipelines
Delimiter Smuggling	Smuggling control markers to terminate data scopes	Delimiter sanitization and strict token budget enforcement
Semantic Poisoning	Injecting falsified information into vector databases	SHA-256 data checksum validation and source provenance tags

2.6 Welford's Online Statistical Algorithm

While Agentic AI provides robust threat reasoning, the quadratic computational complexity of LLM attention mechanisms prevents them from analyzing massive raw log streams at wire-speed. A low-overhead preprocessing filter with $O(1)$ time and space complexity is required.

Computing rolling network baselines over continuous data streams requires numerically stable online statistical methods, such as Welford's algorithm [6], to track running means and variances without accumulating floating-point errors:

$$\mu_k = \mu_{k-1} + \frac{x_k - \mu_{k-1}}{k} \tag{2.1}$$

$$M_{2,k} = M_{2,k-1} + (x_k - \mu_{k-1})(x_k - \mu_k) \tag{2.2}$$

$$\sigma_k^2 = \frac{M_{2,k}}{k-1} \tag{2.3}$$

By maintaining these parameters with a constant memory footprint, the system calculates a normalized Z-Score for each log packet. This enables the rapid elimination of benign network traffic, bypassing the LLM latency bottleneck while preserving the contextual event chains of low-and-slow attacks.

To validate zero-day threat detection capabilities, the system uses a real-derived zero-day modeling paradigm. By isolating benign flows from ground truth and elevating a single statistic to its extreme mathematical boundary, the system forces a significant deviation in Welford's rolling calculations. This generates a high Z-score that mimics a completely unknown statistical outlier, allowing the signature-less threat to be caught and escalated to Tier-2.

2.7 Related Work: Agentic AI for Security Operations

The convergence of Large Language Models and security automation has produced a rapidly growing body of agentic-AI frameworks for the SOC. CyberRAG [13] orchestrates a pool of fine-tuned classifiers through an agentic retrieval-and-reason loop to label and explain web attacks (SQLi, XSS, SSTI), prioritizing classification accuracy and interpretability. LanG [14] proposes a governance-aware platform built on LangGraph that unifies incident correlation, LLM-driven rule generation, and a three-phase attack reconstructor under an explicit AI governance policy engine with human-in-the-loop checkpoints. The Splunk-integrated framework of Sahay et al. [15] couples a reconstruction autoencoder and deep reinforcement learning for initial triage with an LLM for contextual threat hunting inside an enterprise SIEM. On the adversarial side, Pandey and Bhujang [12] do not propose a defensive system but instead formalize the log-substrate prompt-injection threat (the S1–S4 taxonomy above) and empirically demonstrate its effectiveness against LLM-augmented security operations.

Despite their individual strengths, these systems share a structural blind spot: they route raw or lightly filtered telemetry directly into LLM inference, inheriting both the latency bottleneck of multi-billion-parameter attention and, critically, exposure to the very log-substrate injection that Pandey and Bhujang formalize. None couples a wire-speed statistical pre-filter with cognitive-security guardrails, local data sovereignty, and tamper-evident auditing in a single pipeline. Table 4 positions SENTINEL against these works across the capabilities most relevant to a deployable, security-hardened SOC engine.

As summarized in Table 4, SENTINEL is distinguished not by any single component but by their integration: a deterministic Tier-1 filter absorbs the high-velocity benign majority at wire-speed, while a cognitively isolated Tier-2 agent performs grounded reasoning behind cryptographic guardrails. This combination directly addresses the latency–security trade-

Table 4. Comparative analysis of SENTINEL against recent agentic-AI security frameworks. (✓ = supported; ∼ = partial; × = absent; – = not applicable). (Source: Author)

Capability	**SENTINEL** (ours)	**Cyber-RAG** [13]	**LanG** [14]	**Splunk-LLM** [15]	**Watch-tower** [12]
Primary focus	2-tier defense	Attack classif.	Unified govern.	Threat hunting	Attack study
Wire-speed $O(1)$ statistical pre-filter	✓	×	×	∼	–
Cognitive-security guardrails (anti log-substrate injection)	✓	×	∼	×	–
Local / air-gapped inference	✓	∼	∼	×	–
Multi-stage APT / kill-chain correlation	✓	×	✓	∼	–
Hybrid dense+sparse RAG (RRF)	✓	∼	×	×	–
Tamper-evident audit (HMAC chaining)	✓	×	∼	×	–

off that the surveyed frameworks leave open.

2.8 Chapter Summary

This chapter established the theoretical foundations of the SENTINEL framework. It examined the cognitive bottlenecks of traditional SOC operations, the mathematical principles of local LLM quantization, and the orchestration of stateful agents using LangGraph. Additionally, it detailed hybrid search optimization via Reciprocal Rank Fusion, the mechanisms of log-substrate prompt injections and their S1–S4 taxonomy, and the application of Welford's algorithm for low-latency zero-day anomaly detection. A comparative analysis against recent agentic-AI security frameworks (CyberRAG, LanG, Splunk-LLM, and the Watchtower threat study) further positioned SENTINEL as the only approach that unifies wire-speed statistical pre-filtering, cognitive-security guardrails, local data sovereignty, and tamper-evident auditing. Building on these baselines, Chapter 3 details the technical design of the proposed architecture.

Chapter 3

System Design and Technical Implementation

This chapter details the architectural design, mathematical formulations, and software engineering implementation of the SENTINEL two-tier cognitive security framework. Structured as a hybrid threat detection and incident response pipeline, this system is engineered to bridge the performance gap between wire-speed network packet processing and resource-intensive Large Language Model (LLM) reasoning, while establishing robust cognitive guardrails to isolate the LLM from adversarial manipulation. The content analyzes the overall system block diagram, the online statistical pre-filtration algorithms, the state-graph cognitive orchestration workflow, the Dual-RAG retrieval mechanism, the Threat Memory SQLite schema, the defensive guardrail pipeline, the HMAC-secured audit logs, the Streamlit user interface, and the Docker deployment model.

3.1 Overview of the Two-Tier Cognitive Architecture

To protect enterprise environments without incurring prohibitive computational latencies, the SENTINEL system splits the threat detection and response pipeline into two distinct, specialized operational tiers. This architectural separation of concerns is illustrated by the holistic system block diagram in Figure 4.

The proposed architecture divides responsibilities across two main tiers. Tier-1, the stateless wire-speed filtration layer, is situated directly at the network boundary, where it ingests high-velocity telemetry such as flow logs and packet metadata and either computes real-time statistical deviations through Welford's algorithm or applies regex signatures, dropping or logging benign events instantly at wire-speed. Tier-2, the stateful cognitive-reasoning layer, is driven by an autonomous LangGraph agent [8]: it performs deep cognitive reasoning, retrieves grounding contexts from the Dual-RAG engine, correlates logs against the SQLite-based Threat Memory, and issues structured mitigation recommendations such as dynamic firewall blocks. This hybrid layout resolves the LLM latency bottleneck by offloading the vast majority of benign traffic at Tier-1—quantified empirically in

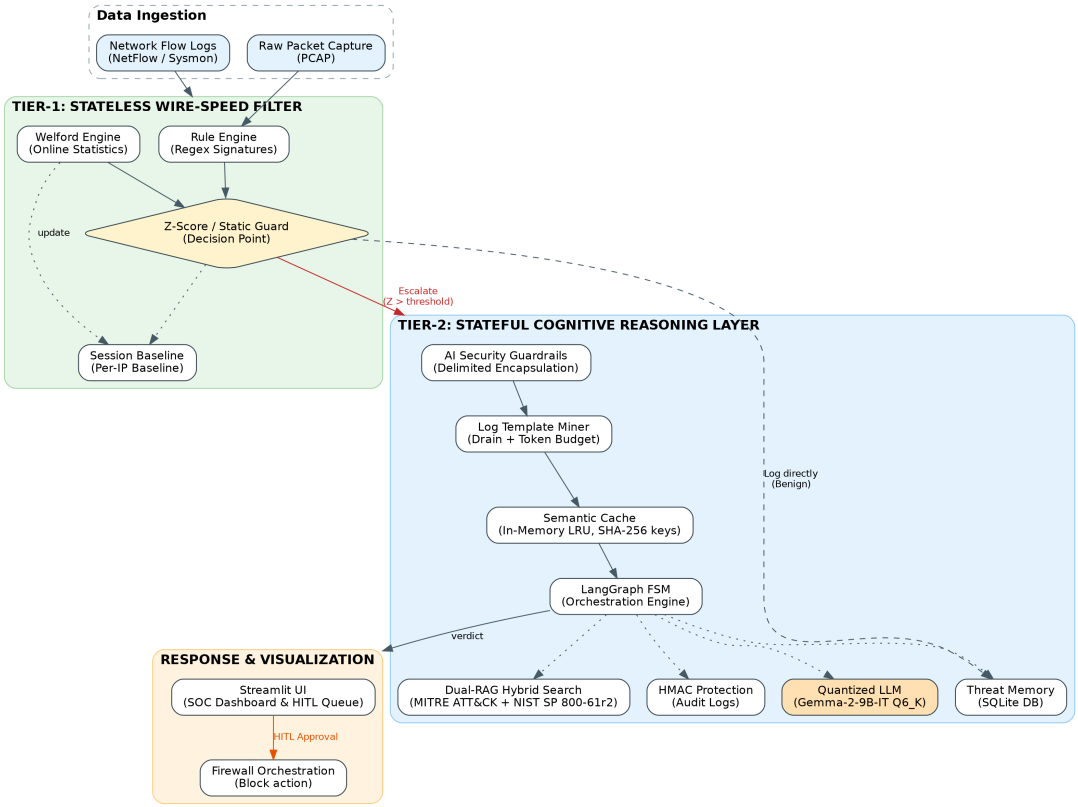

Figure 4. The holistic architecture of the SENTINEL system, illustrating the two-tier data flow and structural components. (Source: Author)

Chapter 4—reserving deep LLM inference solely for complex threat investigations.

The detailed step-by-step runtime flow of the data processing and decision-making logic throughout the system is represented in the system execution flow diagram shown in Figure 5.

As shown in the runtime flow, the process starts immediately when a network flow log is captured. Tier-1 intercepts it and applies signature matching via the Rule Engine; a WAF or injection signature match yields a direct `BLOCK_IP` or `ESCALATE` verdict, while whitelisted traffic is dropped. Otherwise, the numeric statistics are updated and a Z-Score is computed. Benign logs are dropped or written to the local decision log, whereas logs whose risk score exceeds the threshold are escalated. Tier-2 then processes the escalated batch by first compressing and nonce-encapsulating it—running the Drain log parser [20] for event template extraction—before *unconditionally* retrieving grounding context from the Dual-RAG knowledge base (accelerated by a Semantic Cache for recurring templates) together with long-term Threat Memory. The LangGraph agent subsequently invokes the LLM triage node to emit a single structured verdict. Once resolved, the output is validated, sanitized, and written to the database with HMAC chain security, executing firewall blocks or routing the alert to the Streamlit HITL dashboard.

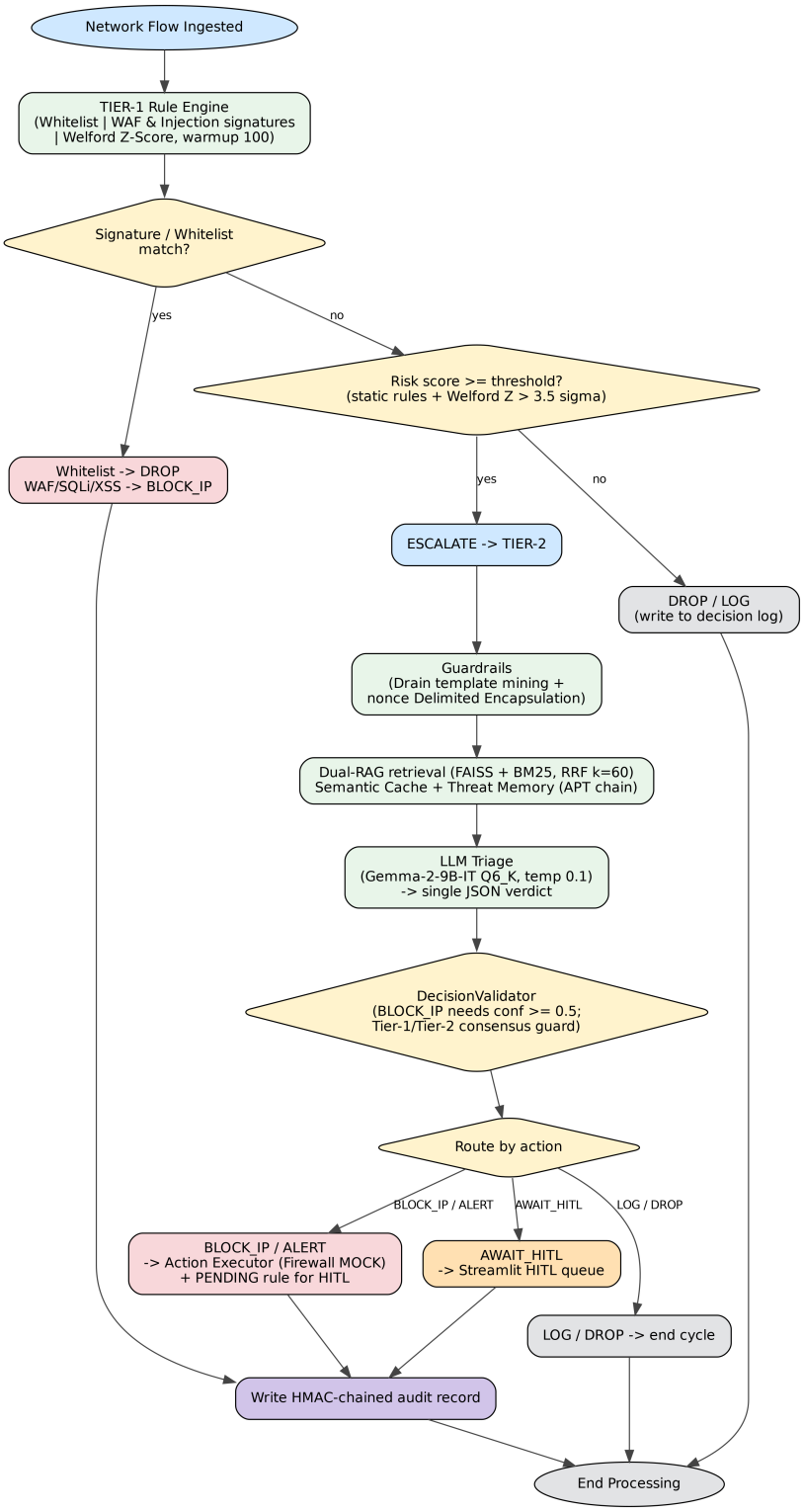

Figure 5. Detailed runtime execution flow and data processing logic within the SENTINEL pipeline. (Source: Author)

3.2 Tier-1: Stateless Wire-Speed Filter (RuleEngine & Welford)

The Tier-1 filtration engine is responsible for ingestion and real-time processing of high-velocity log streams. To manage connection state without incurring disk I/O bottlenecks, the engine incorporates a temporal session baseline manager that maps active source IP addresses. For each active IP session, the system computes statistical traffic features using Welford's online algorithm [6]. The running mean μ_k and sum-of-squared-differences $M_{2,k}$ recurrences introduced in Section 2.6 (Equations 2.1–2.3) yield a numerically stable rolling variance σ_k^2 over a continuous stream of numerical log attributes (such as flow duration, packet counts, or byte size). For each incoming log payload, the system then computes the Z-Score Z of these characteristic traffic attributes:

$$Z = \frac{x_k - \mu_k}{\sigma_k} \tag{3.1}$$

If the Z-Score exceeds a critical threshold (defined by config, e.g., $\alpha > 3.5\sigma$), the packet is labeled with an 'ESCALATE' action and forwarded to Tier-2. To preserve the temporal event chains of low-and-slow campaigns, the system continues updating session baselines. Alongside Welford-based statistical checks, Tier-1 runs a static `RuleEngine` that performs fast regex signature matching to instantly intercept known command patterns (such as SQL injection syntax) and enforce static `DROP` or `BLOCK_IP` actions, bypassing Tier-2.

In the software implementation, Tier-1 is written in Python utilizing two primary classes: `RunningStats` and `SessionBaseline`. The `RunningStats` class maintains the running counts, mean, and sum of squares in memory, executing the mathematical updates in O(1) time complexity. To prevent memory exhaustion from tracking an unbounded number of unique IP sessions, the `SessionBaseline` class structures mapping via a hash table with a garbage-collection daemon that periodically purges inactive sessions based on a configurable Time-To-Live (TTL) parameter. The static signature matches are handled efficiently by compile-time pre-compiled regex objects in Python's `re` library.

3.3 Tier-2: LangGraph Stateful Cognitive AI Agent

Escalated anomalous flows are received by the Tier-2 cognitive engine, modeled as a stateful Directed Acyclic Graph (DAG) compiled by LangGraph [8] and functioning as a Finite State Machine (FSM). The session data and reasoning context are carried within a central state object, `SentinelState`. The compiled graph chains a guardrails node (compression and delimited encapsulation), a RAG-context node, and an LLM-triage node in sequence, terminating in one of two action nodes (an action executor or a human-in-the-loop handler). The RAG-context node *unconditionally* retrieves grounding context from

the hybrid vector-keyword knowledge base before triage, rather than being reached through a knowledge-deficit branch. The LLM-triage node then evaluates the escalated batch using a locally deployed quantized Gemma 2 language model [21] and emits a single structured verdict. The graph routes deterministically on this validated action—high-severity verdicts proceed to the action executor, ambiguous or low-confidence verdicts are diverted to the human-in-the-loop queue, and benign verdicts terminate the cycle—without looping back. The routing states and transitions are outlined in Table 5.

Table 5. State Transition Matrix of the Tier-2 Cognitive Agent FSM. (Source: Author)

Current State	**Input / Condition**	**Next State**	**Output Action**
`GUARDRAILS`	Escalated batch ingested	`RAG_CONTEXT`	Compress & nonce-encapsulate logs
`RAG_CONTEXT`	Grounding context retrieved	`LLM_TRIAGE`	Append MITRE/ NIST context
`LLM_TRIAGE`	action $\in$ {`BLOCK_IP`, `ALERT`}	`ACTION_EXEC`	Enforce block / raise alert
`LLM_TRIAGE`	action = `AWAIT_HITL` (or `BLOCK_IP` with confidence < 0.5)	`HITL`	Queue for SOC analyst
`LLM_TRIAGE`	action $\in$ {`LOG`, `DROP`}	`END`	Close cycle
`ACTION_EXEC` / `HITL`	Action completed	`END`	Write HMAC audit record

Grounding the triage node in the retrieved MITRE/NIST documents constrains the LLM to verified context, mitigating hallucination. The triage node emits a structured JSON payload detailing the threat class, a confidence score, and recommended block rules. High-confidence malicious verdicts are dispatched to the action executor, whereas low-confidence or explicitly ambiguous verdicts are routed to the Human-in-the-Loop (HITL) queue for analyst review. A deterministic `DecisionValidator` additionally demotes any `BLOCK_IP` verdict whose confidence falls below 0.5 to `AWAIT_HITL`, and overrides the LLM to `AWAIT_HITL` whenever it contradicts a Tier-1 attack signal (the tier-consensus guard against semantic social-engineering).

The technical implementation of the state graph utilizes LangGraph's compiler. The state

is represented as a structured Python typed dictionary containing append-only fields like `extracted_iocs` and `decisions` to prevent semantic drift across the triage pipeline. The LLM connection is managed via the `LLMClient` class connecting to a local quantized llama.cpp service running the Gemma 2 model [21], configured with strict parameters including `temperature=0.1` and a fixed 8,192-token context window to ensure near-deterministic reasoning and prevent resource depletion.

3.4 Dual-RAG Mechanism and Semantic Memory

The Dual-RAG mechanism grounds the agent's decisions by retrieving contexts from two unified domain-specific databases: the MITRE ATT&CK knowledge base [22], which maps threat behaviors, tactics, and techniques (TTPs), and the NIST SP 800-61r2 playbook repository, which provides standard incident handling procedures [10]. Rather than relying on simple keyword matching or dense semantic vectors in isolation [7], retrieval utilizes a hybrid search strategy. A comparison between the dense and sparse search components is detailed in Table 6.

Table 6. Comparison of Retrieval Paradigms in the Hybrid Search Strategy. (Source: Author)

Dimension	**Dense Vector Search (FAISS)**	**Sparse Keyword Search (BM25)**
Underlying Model	all-MiniLM-L6-v2, 384-dim embeddings [23]	BM25 term weighting
Matching Logic	Cosine Semantic Similarity [18]	Term Frequency-Inverse Document Frequency [19]
Strengths	Captures synonyms, concepts, and abstract semantics	Matches exact technical IDs (CVEs, IPs, port numbers)
Limitations	Weak at exact keyword matching of technical hashes	Blind to semantic synonyms and conceptual context

The scores from the dense (FAISS) and sparse (BM25) pipelines are consolidated using Reciprocal Rank Fusion (RRF) [11], which calculates a unified score R for each document d over the set of search runs M:

$$R(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \tag{3.2}$$

where $r_m(d)$ is the rank of document d in search run m, and k is a constant scaling param-

eter (typically $k = 60$). The three top-ranked passages from this fused list are forwarded to the triage node as grounding context. To avoid redundant embedding and search overhead, a Semantic Cache module sits before the retrieval pipeline. The cache derives a deterministic key by computing the SHA-256 digest of the mined log template, so escalated logs that share the same attack template reuse a previously computed retrieval result instead of re-embedding and re-searching, reducing per-query processing time from hundreds of milliseconds to microseconds. The cache is implemented as an in-memory Least Recently Used (LRU) structure (a Python `OrderedDict`) bounded to 500 entries, with an 1800-second Time-To-Live (TTL) that evicts stale templates.

3.5 Threat Memory and APT Campaign Correlation

Stealthy threat actors frequently split their attack chains into low-severity events scattered across long temporal windows, bypassing stateless packet filters. To expose these low-and-slow campaigns [4], the SENTINEL architecture incorporates a persistent Threat Memory module, implemented as a local SQLite database. The module is structured around specialized schemas to track historical alerts, entity reputations, and temporal attack sequences. The database schemas are detailed in Table 7.

The reputation score of an IP address escalates based on the severity of the alerts it triggers, up to a maximum cap of 100. Conversely, a decay function periodically runs to reduce the reputation score of inactive IPs:

$$S_{new} = S_{old} \times (1 - \lambda)^t \tag{3.3}$$

where λ is the decay rate and t is the inactive time elapsed. An APT correlation routine scans the `threat_events` table. If a single source IP triggers anomalies across at least two distinct days (optionally spanning multiple kill-chain phases), the system automatically elevates the threat classification to high-severity, treating it as an emergent multi-day campaign rather than an isolated event. In the implementation, these temporal intervals and reputation increments are computed by Python-side upsert and aggregation queries over the SQLite store, avoiding the need to hold long-term state in the agent's working memory.

3.6 AI Security Guardrails and Encapsulation Layer

Because LLMs are vulnerable to instruction hijacking, the SENTINEL system implements a multi-layered AI Security Guardrails layer to isolate the reasoning engine from untrusted inputs [9]. The core defense mechanism relies on Delimited Data Encapsulation. For

Table 7. Threat Memory Database Schema Specifications. (Source: Author)

Table Name	**Primary Fields**	**Functional Purpose**
`ip_reputation`	`ip` (PK), `reputation_score`, `total_incidents`, `total_blocks`, `first_seen`, `last_seen`, `last_mitre_technique`	Tracks the long-term reputation score (0–100) and incident history of source IPs, subject to time decay.
`known_entities`	`id` (PK), `entity_type`, `entity_value` (unique), `description`, `is_active`	Pre-seeds verified internal tools/services (e.g., AD, scanners) to suppress false positives.
`threat_events`	`id` (PK), `src_ip`, `dst_ip`, `apt_phase`, `apt_day`, `label`, `timestamp`	Records individual escalated events for emergent multi-day APT chain correlation.
`apt_indicators`	`id` (PK), `indicator_type`, `indicator_value`, `confidence`, `occurrence_count`, `mitre_chain`	Stores confirmed APT correlation indicators for long-term campaign monitoring.

each session, the system generates a cryptographically secure, randomized hexadecimal token (e.g., <|DATA_8F3E|>) using Python's `secrets.token_hex()` routine (built on `os.urandom`). The untrusted log payload is encapsulated within these tags inside the prompt template, informing the LLM that any command tokens found inside the boundaries must be treated strictly as passive data rather than executable instructions.

To defend against Token Denial-of-Service (DoS) and context overflow attacks, the system incorporates a Log Template Miner based on the Drain log parsing algorithm [20]. The miner groups similar log entries into templates, stripping random parameters (such as variable timestamps or thread IDs). Operating within a 4,000-token budget, the Token Budget Manager allocates up to 60% of this budget to compressed templates and 40% to high-entropy outliers. Finally, an Output Sanitization Filter enforces strict JSON schema conformance, verifying that the LLM response contains only allowed fields and stripping markdown image tags, SVG scripts, and hex-encoded bypass vectors [12]. The sanitization is implemented using the `output_sanitizer` module which intercepts LLM outputs before they are processed by the database or dashboard.

3.7 Data Integrity and HMAC Protection

To secure the security log database against tampering by malicious insiders or attackers who gain access to the file system, the system implements HMAC log chaining [29]. Each security log entry is cryptographically linked to the preceding record, forming a private blockchain-like audit trail. For log entry i, the record hash H_i is computed as:

$$H_i = \text{HMAC-SHA256}(D_i \parallel H_{i-1}, K) \tag{3.4}$$

where D_i is the payload data of the current log, H_{i-1} is the hash of the previous log record, and K is the secret HMAC key stored securely. If an attacker attempts to alter a log label or delete an entry, the hash chain breaks, and the validation checks run by the dashboard immediately flag the tampering. Additionally, the system protects the RAG vector database against semantic poisoning attacks. The vector database files are monitored by a checksum manager that calculates and compares SHA-256 hashes of the files before loading them into memory, ensuring that no malicious documents have been injected into the agent's retrieval memory.

The integrity chaining is implemented in the audit-logging routine, while an `Action-Validator` class enforces an allowlist of valid security actions, detecting and rejecting any shell metacharacters inside the fields of incoming payloads to prevent remote code execution (RCE) attempts during firewall script orchestration.

3.8 SOC Administration Dashboard Implementation (Streamlit UI)

The Streamlit-based SOC Administration Dashboard follows a reactive model where the Streamlit frontend communicates dynamically with the database and files of the backend to update UI elements. The dashboard integrates three specialized display modules. First, a real-time queue interface visualizes Tier-1 telemetry throughput and Tier-2 analysis verdicts. Second, a threat memory analytics panel plots APT alert propagation graphs across IP-based temporal timelines. Third, an audit trail integrity module verifies the HMAC chain in the database, displaying status indicators where a green light signals a valid chain and a red light warns that logs have been modified. Additionally, the dashboard incorporates a Human-in-the-Loop interface, allowing analysts to manually review and either approve or reject the agent's recommended firewall blocks, updating live firewall configurations on-the-fly.

3.9 Containerization and Infrastructure Deployment

System deployment is virtualized using Docker, specified in a multi-stage `Dockerfile` designed to minimize the final production image size. The build process compiles llama.cpp with CUDA support to enable GPU-accelerated local inference within the container. Infrastructure services are managed via `docker-compose.yml`, which links the agent reasoning container with the Streamlit dashboard container, sharing log and database directories using persistent volume mounts. This ensures that the `threat_memory.db` and `guardrails_audit.db` SQLite databases remain decoupled from transient container lifetimes.

3.10 Scalability and Graceful Degradation

A natural objection to any LLM-in-the-loop pipeline concerns throughput: if telemetry arrives at billions of events per day, would the language model not become an insurmountable bottleneck? The two-tier design answers this by treating the two tiers as fundamentally different scaling regimes, rather than by provisioning ever-larger inference hardware.

Tier-1 is a throughput problem, not a latency wall. Because each Welford update and regex match executes in $O(1)$ time and the per-IP session state is mutually independent, Tier-1 is embarrassingly parallel: the workload shards cleanly across cores and hosts by source-IP hash. In the Python prototype each decision takes roughly $0.6\,\text{ms}$, so a single core sustains on the order of a thousand events per second; a throughput of one billion events per day averages about 1.2×10^4 events per second, which is within reach of a

modest multi-core node, with bursts absorbed by horizontal sharding. A production implementation in compiled or stream-processing form (for example, an eBPF probe or a Rust stream operator) raises the per-core ceiling by orders of magnitude. The $O(1)$ algorithm is therefore what makes wire-speed attainable; the prototype validates the filtering ratio and correctness rather than line-rate throughput.

The cognitive tier never sees the raw escalation count. Although Tier-1 forwards only the anomalous minority (empirically about 0.2% of benign-dominated traffic), three further layers collapse this figure before any inference occurs. First, the Drain template miner compresses near-identical log lines—the repetitive bulk of real attacks such as scans, brute-force attempts, and volumetric floods—into a small set of templates. Second, the Semantic Cache, keyed on the SHA-256 digest of each template, returns a previously computed verdict for any recurring template without re-invoking the model. Third, once an IP is blocked, its Threat Memory reputation record causes subsequent events to be dropped at Tier-1 rather than re-escalated. The effective inference load is therefore bounded not by the raw event volume but by the number of *genuinely novel* anomaly templates, which is far smaller and is processed in batches.

The batch, not the event, is the unit of inference. Escalated logs are accumulated into a single cycle batch and submitted to the model in *one* inference call rather than one call per event; the number of logs carried per batch is bounded only by the token budget, so dozens of compressed escalations are adjudicated by a single ($\approx$5.7 s) inference. Equally important, the cognitive tier sits *off* the synchronous blocking path: Tier-1 has already issued its deterministic `BLOCK_IP` or `DROP` verdict for unambiguous attacks at wire-speed, so the LLM batch contributes explanation, MITRE/NIST grounding, and cross-event correlation *asynchronously*. A backlog in the cognitive tier therefore delays enrichment, not protection—the actual mitigation of clear threats is never gated on LLM latency. The sustained deep-reasoning throughput of a single accelerator is thus (logs per batch) divided by (per-batch latency), which exceeds a naive per-event rate by the batch factor, before any cache hits are counted.

Behaviour under an adversarial flood. The worst case for the cognitive tier is a flood of anomalies that are all novel, so that template compression and caching cannot absorb them. Two design principles bound the damage. First, such a flood is itself a Tier-1-detectable signal: the surge in event rate is a Welford outlier that triggers a coarse volumetric mitigation—for example, rate-limiting or blocking the offending address ranges—without per-event reasoning. Second, the cognitive tier is a bounded-capacity resource that must shed load gracefully: when its queue saturates, the pipeline falls back to the deterministic Tier-1 verdict (block or alert from the Z-score and rule engine), deferring only the richer LLM explanation and grounding. Security coverage is thus preserved under overload, and only cognitive enrichment degrades. Consequently, GPU capacity is provisioned

for the steady-state trickle of novel anomalies—which a single consumer-grade accelerator already serves in this work—rather than for the worst-case line rate, since scaling the model to match raw ingestion would be both architecturally self-defeating and economically irrational.

It must be stated honestly that the present prototype implements the load-shedding ratio and the caching layers but not yet an explicit backpressure-and-degradation controller; under a sustained novel-anomaly flood it would saturate the escalation queue, as acknowledged among the limitations in Chapter 5. Formalizing this graceful-degradation policy, together with the horizontal GPU scaling and multi-agent decomposition outlined in future work, is the natural path toward a production-grade deployment.

3.11 Chapter Summary

This chapter has established the comprehensive system design and technical implementation details of the SENTINEL security architecture. By merging wire-speed stateless filtration (Tier-1) with stateful cognitive agentic reasoning (Tier-2) and wrapping the entire system in robust cryptographic and AI-specific guardrails, the framework effectively balances high throughput with secure deep reasoning. The Streamlit administration dashboard bridges this automated system with human oversight via the HITL queue, and the multi-stage Docker setup ensures isolated, reproducible deployments. Chapter 4 will delineate the empirical evaluation, presenting performance logs, ablation studies, and statistical test results verifying the capabilities of this merged architecture.

Chapter 4

Experiments and Evaluation

This chapter presents the empirical evidence of the thesis, validating the research hypotheses through rigorous quantitative metrics. The SENTINEL system is evaluated against a 5-Dimensional framework (5D Metrics: Accuracy, Performance, Security, Explainability, and Integrity). The content emphasizes the layered Ablation Study design and the application of non-parametric statistical models (McNemar's Test, Mann-Whitney U Test) to prove that the architectural enhancements are statistically significant and mathematically sound.

4.1 Experimental Environment and Datasets

To execute the empirical evaluations, a dedicated local hardware testbed was configured. The hardware setup consists of an Intel Core i7 CPU (specifically the i7-14700KF model), 32GB of DDR5 RAM, and an NVIDIA RTX 4060 Ti 16GB GPU, which was utilized to execute the local quantized *Gemma-2-9B-IT* Q6_K model. The evaluations rely on two major datasets: the CSE-CIC-IDS2018 dataset [1], which encompasses diverse network attack typologies—including Brute Force, DoS, and Web Attacks—serving as the baseline for evaluating classification accuracy; and the DAPT2020 dataset [25], which is employed for testing the system's capacity to correlate multi-stage Advanced Persistent Threat (APT) campaigns over prolonged timelines. To replicate the properties of a live SIEM environment, a Unified Data Stream simulator was scripted to merge and replay the network logs chronologically according to virtual timestamps, feeding them to the detection queues in real time.

A detailed overview of the CSE-CIC-IDS2018 attack labels, their mapped MITRE ATT&CK techniques, the default mitigation actions configured in the SENTINEL pipeline, and the source raw data files is summarized in Table 8.

Table 8. CSE-CIC-IDS2018 Attack Labels, MITRE ATT&CK Mapping, and Default Response Actions. (Source: Author)

Attack Category (Label)	**MITRE ATT&CK**	**Default Action**	**Source CSV File**
Benign	None	`LOG`	Wednesday-14-02-2018
SSH-Bruteforce / FTP-BruteForce	T1110	`BLOCK_IP`	Thursday-22-02-2018
DoS attacks-Hulk / Slowloris	T1499	`ALERT`	Friday-16-02-2018
DoS attacks-GoldenEye / SlowHTTPTest	T1499	`ALERT`	Thursday-15-02-2018
DDOS attack-HOIC / LOIC-UDP	T1499	`ALERT`	Thuesday-20-02-2018
Brute Force -Web / -XSS	T1110 / T1059	`BLOCK_IP` / `ALERT`	Friday-23-02-2018
SQL Injection	T1190	`ALERT`	Friday-23-02-2018
Infiltration	T1078	`AWAIT_HITL`	Thursday-01-03-2018
Bot	T1071	`ALERT`	Friday-02-03-2018

4.2 5D Evaluation Metrics and Ablation Study Design

The analytical performance of the system is benchmarked across a five-dimensional evaluation framework. These 5D metrics include Accuracy, measured by Precision, Recall, and F1-Score; Performance, measuring latency and throughput; Security, defined by the adversarial robustness rate; Explainability, evaluated via audit completeness; and Integrity, verified by HMAC validation. To isolate the quantitative impact of each proposed component, an ablation study was designed across six specific configurations. Configuration A (the traditional baseline) relies exclusively on a rule-based signature engine at the ingestion layer, without any LLM routing. Configuration B (the pure-LLM baseline) submits every network log to local LLM inference, without Welford pre-filtering, RAG context, or security guardrails. Configuration C (Welford-filtered) inserts Welford's Tier-1 statistical filter ahead of the LLM so as to reduce the ingestion volume. Configuration D (single-RAG grounded) adds a single dense vector-search database (FAISS with MiniLM embeddings) to the filtered LLM triage. Configuration E (hybrid-RAG grounded) expands this retrieval component into a hybrid of dense vector search (FAISS) and sparse keyword search (BM25). Finally, Configuration F (the holistic SENTINEL) combines the Tier-1 Welford filter, the stateful Tier-2 LangGraph agent, hybrid Dual-RAG (FAISS + BM25)

consolidated via RRF, and the defensive guardrails. All six configurations were evaluated empirically on a stratified 300-sample subset of the labeled ground truth, so that the marginal contribution of each added component—the language model itself, the Welford pre-filter, dense retrieval, hybrid retrieval, and the guardrail stack—can be isolated from the difference between consecutive configurations.

Before the results are presented, two conventions are fixed so that the figures remain interpretable. **Metric-to-question mapping.** SENTINEL is a two-tier filter-and-reason pipeline rather than a monolithic binary classifier; accordingly, each research question is judged by the metric that actually measures it, and binary F1 is treated as a secondary diagnostic of the Tier-1 gate's operating point rather than as the headline number. RQ1 (wire-speed filtration) is assessed by the benign-filtration and escalation rates together with Tier-1 latency; RQ2 (secure, low-latency triage) by the adversarial block rate, the reasoning-quality scores, and the Tier-1/Tier-2 latency separation; and RQ3 (grounding, memory, and validation) by the RAGAS faithfulness and relevancy scores and by the emergent APT correlation. **Prediction convention.** A single positive rule is applied uniformly throughout: a flow is counted as a positive detection whenever the system does not silently discard it—that is, whenever Tier-1 issues an actioning or escalating verdict (BLOCK_IP, ALERT, AWAIT_HITL, or ESCALATE). **Granularity and headline.** Two evaluation granularities appear and are never mixed within a single comparison. The realistic *per-event* Unified-Stream benchmark (Table 9, $\mathrm{F1} = 0.594$) replays every event individually through a clean pipeline and is taken as the honest accuracy headline; the *per-sample* ablation (Table 13) instead groups each labelled incident into one batched verdict on an attack-dominated subset, where the resulting $\mathrm{F1} = 0.967$ equals the base rate of that subset and is therefore used only to isolate per-component latency, never as an accuracy claim.

4.3 Classification Accuracy on the Unified Stream

The primary accuracy evaluation was conducted on the Unified Stream—a single time-ordered merge of 150 benign warm-up flows and 4,526 main-stream events (CSE-CIC-IDS2018 attacks and benign traffic, DAPT2020 APT events, and seven real-derived zero-day outliers)—processed end-to-end with a freshly initialized Threat Memory. Table 9 reports the binary classification performance of the Tier-1 gate on this stream.

The Tier-1 gate attains high precision (0.939) at a moderate recall (0.435). This recall figure is a deliberate design property rather than a deficiency: Tier-1 blocks only attacks that are unambiguous at the network layer and *escalates* subtler signals to the Tier-2 agent, intentionally trading raw recall for a low false-positive load and for the preservation of low-and-slow event chains. The end-to-end recovery of these escalated cases is quantified by the APT, zero-day, and ablation analyses that follow.

Table 9. Tier-1 binary classification on the Unified Stream (4,526 events). (Source: Author)

Metric	Value
Precision	0.939
Recall (attack)	0.435
F1-score	0.594
Accuracy	0.448
TP / FP / TN / FN	1724 / 113 / 187 / 2243

4.4 Multi-Stage APT Detection (Emergent Correlation)

The APT-correlation capability was evaluated on the DAPT2020 chains embedded in the same stream, with the Threat Memory initialized *empty* so that every verdict must emerge from accumulated multi-day evidence rather than from a pre-loaded answer key. Table 10 summarizes the outcome.

Table 10. Emergent multi-stage APT detection on DAPT2020 chains (clean memory). (Source: Author)

Metric	Value
Ground-truth APT IPs present in stream	3
Detected	3
Missed	0
Recall	1.00
Average detection lag	8.33 events

All three ground-truth APT campaigns were detected (recall = 1.00) with an average detection lag of only 8.33 events, confirming that multi-day lateral-movement chains surface from the persistent Threat Memory even when each constituent event is individually low-signal at Tier-1.

Two complementary checks bound the strength of this result. First, because only three multi-day campaigns are present in the stream, the point estimate is reported with its Wilson 95% confidence interval: a recall of $3/3$ corresponds to $[0.44, 1.00]$, so the figure should be read as indicative of correct emergent correlation rather than as a precise population estimate. Second, a negative control verifies that the detector does not merely fire on any address seen across several days: of the *four* non-APT source IPs that nonetheless appear on two or more distinct days in the stream, none triggered an APT verdict (specificity $= 1.00$, zero false firings). This separation arises because discrimination is enforced at the recording gate—only events the pipeline flags as malicious are written to the Threat

Memory—so a benign address accumulates fewer than two attack-days and never satisfies the multi-day chain condition. The capability is thus genuine but modest in statistical power; a larger multi-campaign corpus is identified as future work.

4.5 Zero-Day Detection via Welford Anomaly Scoring

Seven signature-less zero-day outliers were injected into the stream by elevating a single statistic of a genuine benign flow to its extreme boundary. Table 11 contrasts the static rule engine against the online Welford Z-score filter.

Table 11. Zero-day (signature-less) detection: static rule engine vs. Welford Z-score. (Source: Author)

ID	Welford Z-score	Static engine	SENTINEL (Tier-1+2)
ZD-001	22,075.95	DROP	AWAIT_HITL
ZD-002	7.52	DROP	AWAIT_HITL
ZD-003	15,547.21	DROP	AWAIT_HITL
ZD-004	266.02	DROP	AWAIT_HITL
ZD-005	318,508.60	DROP	ESCALATE
ZD-006	118,783.41	DROP	AWAIT_HITL
ZD-007	3,786.94	DROP	ESCALATE
Total caught		**0 / 7**	**7 / 7**

Every zero-day outlier was missed by the signature-based static engine (which issued a DROP) yet caught by the online Welford filter, which flagged each as a statistical anomaly ($Z > 3.5$) and escalated it to Tier-2 for human-in-the-loop adjudication.

4.5.1 Graded Detection Boundary

The seven injected outliers above sit at extreme deviations ($Z \gg 100\sigma$), so a $7/7$ catch rate—though correct—does not by itself locate the detector's sensitivity boundary. To map that boundary, a single Welford-tracked feature of a genuine benign flow was elevated to a controlled deviation of $k\sigma$ for $k \in \{2, 3, 3.5, 4, 5, 6, 8, 10, 20, 50, 100\}$, sweeping 210 probes per level (30 static-clean benign base flows $\times$ 7 features) through a frozen Tier-1 baseline. Table 12 and Figure 6 report the resulting detection curve.

Three regimes are visible. Below 3.5σ the injected deviation is statistically indistinguishable from the benign tail, and detection rests at a floor of 8.6%—the residual rate at which a base flow already contains a naturally anomalous feature. At the 3.5σ operating point the detection rate rises to 21.4%, marking the onset of the transition, and by 4σ it saturates

Table 12. Graded zero-day detection boundary: detection rate versus injected single-feature deviation (Tier-1 only, $n = 210$ probes per level). (Source: Author)

Deviation k (σ)	**Mean realized Z**	**Noticed ($Z > 3.5\sigma$)**	**Escalated $\rightarrow$ Tier-2**
2.0	2.5	8.6%	8.6%
3.0	3.2	8.6%	8.6%
3.5 (op.)	3.7	**21.4%**	**21.4%**
4.0	4.2	**100%**	**100%**
5.0	5.1	100%	100%
10.0	10.0	100%	100%
100.0	100.0	100%	100%

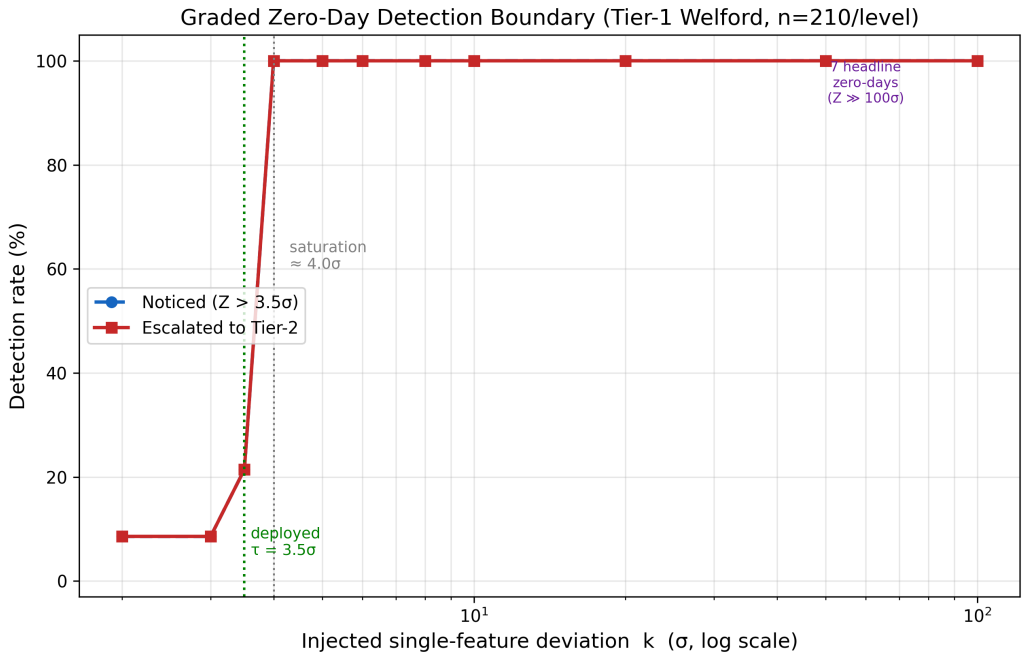

Figure 6. Graded zero-day detection boundary. Detection saturates at 100% once a single-feature deviation reaches 4σ; the deployed 3.5σ threshold sits at the onset of this knee, and the seven headline zero-day samples ($Z \gg 100\sigma$) lie deep in the saturated regime. (Source: Author)

at 100% and remains there for every larger deviation. The detector's effective boundary is therefore $\approx 4\sigma$, with the deployed 3.5σ threshold deliberately placed at the onset of the knee so that incipient anomalies are surfaced early. The seven headline zero-day samples, at $Z \gg 100\sigma$, lie far inside the saturated regime, which is why the coarse $7/7$ result reported above is robust; the graded curve supplies the quantitative boundary that the binary count alone cannot. Notably, the noticed and escalated rates coincide at every level, confirming that on a clean benign substrate a noticed single-feature anomaly is also escalated rather than silently logged.

4.6 Ablation Study: Two-Tier Latency Trade-off and Statistical Tests

The ablation study isolates the contribution of each component by running all six configurations (A–F) on the same stratified 300-sample subset of the labeled ground truth. Two non-parametric tests anchor the comparison between the deterministic and full endpoints: McNemar's test [27] on the paired binary verdicts, and the Mann-Whitney U test [28] on the per-event latency distributions. The results are summarized in Table 13.

Table 13. Cumulative ablation across all six configurations (300 stratified samples). Every configuration yields *identical* binary verdicts (zero discordant pairs vs. A); the components add latency, not classification gain. (Source: Author)

Configuration	**F1**	**Prec.**	**Rec.**	**Escal.**	**Lat./ev.**
A — Tier-1 only (no LLM)	0.967	0.937	1.000	—	0.6 ms
B — Pure LLM (no gate/RAG/guard)	0.967	0.937	1.000	100%	7.29 s
C — Welford gate + LLM	0.967	0.937	1.000	62.7%	3.95 s
D — C + dense RAG (FAISS)	0.967	0.937	1.000	62.7%	5.62 s
E — D + hybrid RAG (BM25+RRF)	0.967	0.937	1.000	62.7%	6.07 s
F — Full SENTINEL	**0.967**	**0.937**	**1.000**	**62.7%**	**5.74 s**

Three findings emerge. First—and most striking—all six configurations produce *identical* binary verdicts on every one of the 300 samples (zero discordant pairs against Configuration A, with $\mathrm{F1} = 0.967$, precision $= 0.937$, and recall $= 1.000$ throughout). Successively adding the language model (B), the Welford gate (C), dense retrieval (D), hybrid retrieval (E), and the full guardrail stack (F) leaves the coarse attack/benign decision unchanged, so McNemar's test is degenerate for every pair ($p = 1.0$, no discordant pairs). The architectural value of these components is therefore *orthogonal* to binary detection: it lies in the contextual reasoning, MITRE/NIST grounding, zero-day escalation, and adversarial robustness quantified elsewhere in this chapter, rather than in raw classification gain. (The stratified subset is 94% attack by construction; because every configuration predicts the positive class on every sample, the shared $\mathrm{F1} = 0.967$ is *exactly* the score of a trivial all-positive classifier—that is, the base rate of this subset—and is consequently a property of

the benchmark rather than a capability of any single configuration. The honest end-to-end accuracy headline is instead the balanced Unified-Stream result of Table 9, $\mathrm{F1} = 0.594$; the present ablation isolates the per-component *latency* and verdict-equivalence comparison rather than classification accuracy.)

Second, where the configurations differ sharply is in *latency*, and the progression isolates the cost of each component. Submitting every event to the LLM (Configuration B) incurs the highest mean latency, 7.29 s; inserting the Welford gate (Configuration C) cuts this to 3.95 s by resolving 37.3% of traffic at wire-speed—escalation falls from 100% to 62.7%—so the pre-filter's contribution is throughput, not accuracy. Adding dense RAG (Configuration D, 5.62 s) and then hybrid BM25 + RRF retrieval (Configuration E, 6.07 s) each introduce retrieval overhead without altering F1, confirming that the retriever's role is grounding quality (Table 17) rather than classification. The full system (Configuration F) averages 5.74 s per escalated event against 0.6 ms at Tier-1, and the Mann-Whitney U test confirms that this Tier-1/Tier-2 latency separation is highly significant ($U = 27{,}053.5,\ p = 2.8 \times 10^{-17}$).

Third, the ablation validates the central design hypothesis: because the expensive cognitive layer does not change the binary verdict on clear-cut traffic, a deterministic wire-speed filter must absorb the bulk of it so that costly reasoning—whose value is qualitative rather than classificatory on this subset—is reserved for the genuinely ambiguous minority. For the 37.3% of traffic resolved at Tier-1, the latency reduction versus full LLM inference exceeds 99.9%. This is precisely the property that makes a multi-second local LLM deployable as a production SOC component: rather than being placed on the critical path of every event—where its latency would be prohibitive (Configuration B)—the model is gated behind an $O(1)$ statistical filter and invoked only for the residual minority, so the architecture's contribution is operational feasibility under cost and latency constraints, not a marginal gain in binary accuracy.

4.6.1 Controlled Balanced Ablation

Because the stratified subset above is attack-dominated, it cannot expose differences in false-positive behaviour. To obtain a discriminating comparison, the six configurations were re-run on a *balanced* subset of 150 benign and 150 attack samples, with the Welford baseline warmed on a disjoint set of 150 held-out benign flows so that genuine benign traffic is correctly calibrated. Table 14 reports the outcome.

On balanced data the configurations no longer collapse onto a single point, and the contrast is informative. The pure-LLM baseline (B) attains the highest recall (0.867) and a nominally higher F1 (0.640 versus 0.552), but only by flagging 126 of the 150 benign flows as malicious: its precision collapses to 0.508, and McNemar's test against every gated con-

Table 14. Controlled balanced ablation (150 benign + 150 attack). Unlike the attack-dominated subset, the configurations now separate: the pure-LLM baseline differs sharply, while the gated configurations (C–F) are verdict-identical. (Source: Author)

Configuration	**F1**	**Prec.**	**Rec.**	**Escal.**	**Lat./ev.**
A — Tier-1 only (no LLM)	0.552	0.627	0.493	—	< 1 ms
B — Pure LLM (no gate/RAG/guard)	0.640	0.508	0.867	100%	6.50 s
C — Welford gate + LLM	0.552	0.627	0.493	9%	0.59 s
D — C + dense RAG (FAISS)	0.552	0.627	0.493	9%	0.85 s
E — D + hybrid RAG (BM25+RRF)	0.552	0.627	0.493	9%	0.95 s
F — Full SENTINEL	**0.552**	**0.627**	**0.493**	**9%**	**0.84 s**

figuration is decisive (138 discordant verdicts in a single direction, $p < 0.001$). Inserting the Welford gate (C) reverses this—by resolving 91% of traffic at Tier-1 and escalating only the 9% residual, it cuts false positives from 126 to 44 (precision $0.508 \rightarrow 0.627$) and mean latency from 6.50 s to 0.59 s, an order-of-magnitude reduction, at the cost of recall ($0.867 \rightarrow 0.493$). The gate's contribution is therefore concrete and measurable: precision and throughput, deliberately traded against recall, exactly as the precision-favoring design intends. Adding dense retrieval (D), hybrid retrieval (E), and the guardrail stack (F) leaves the binary confusion matrix *identical* to C (zero discordant pairs), reaffirming on balanced data the earlier conclusion that the retriever and guardrails contribute grounding, reasoning quality, and security rather than binary-classification gain. The seemingly higher F1 of the pure-LLM baseline is thus an operational mirage: a 49% benign false-positive rate would be unusable under real alert-fatigue constraints, and a 6.5 s cost on *every* event cannot scale, whereas the gated two-tier design preserves precision and runs an order of magnitude faster.

4.6.2 Welford Threshold Sensitivity

A natural objection is whether the 3.5σ Welford escalation threshold was over-fitted to the test data. To address this, the threshold τ was swept across $[2.0\sigma, 5.0\sigma]$ and the Tier-1 layer was re-evaluated—without any LLM—on the same realistic mixed stream used for the classification benchmark of Table 9. Table 15 and Figure 7 report the resulting trade-off.

Three observations follow. First, Tier-1 precision is effectively invariant to the threshold, remaining within the narrow band 0.939–0.945 across the full $[2.0\sigma, 5.0\sigma]$ sweep; the gate's low false-alarm property is thus a structural feature rather than an artifact of one tuned value. Second, all seven signature-less zero-day outliers are caught at every threshold ($7/7$): because a genuine outlier deviates by hundreds of standard deviations, 3.5σ is not a fragile sweet spot chosen to maximize zero-day recall. Third, the threshold behaves as

Table 15. Welford Z-score threshold sensitivity on the real mixed stream (Tier-1 only, LLM-free). The deployed 3.5σ operating point reproduces the headline classification F1 of Table 9. (Source: Author)

τ (σ)	F1	Prec.	Rec.	Benign FP	Escal.	Zero-day
2.0	0.676	0.939	0.529	0.453	0.563	7/7
2.5	0.667	0.943	0.516	0.417	0.552	7/7
3.0	0.663	0.942	0.512	0.417	0.548	7/7
3.5 (op.)	**0.594**	**0.939**	**0.435**	**0.377**	**0.478**	**7/7**
4.0	0.582	0.941	0.422	0.350	0.464	7/7
4.5	0.578	0.942	0.416	0.337	0.459	7/7
5.0	0.576	0.945	0.414	0.317	0.456	7/7

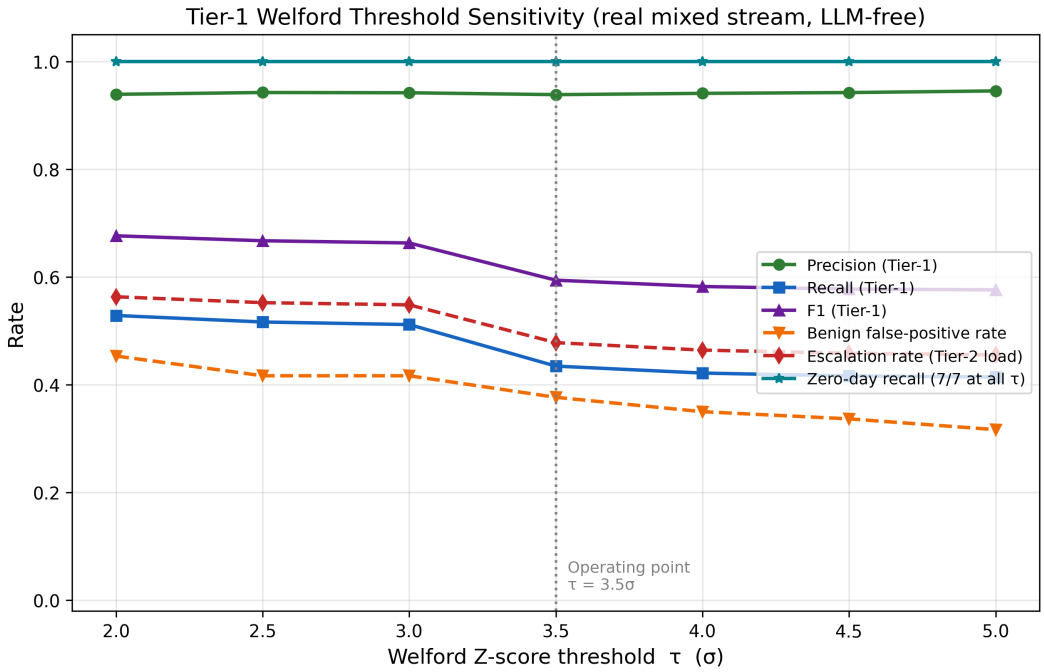

Figure 7. Tier-1 Welford threshold sensitivity. Precision is invariant (0.939–0.945) and zero-day recall is saturated (7/7) across the entire range, while the escalation rate and benign false-positive rate vary smoothly—making τ a tunable load dial rather than a fragile sweet spot. (Source: Author)

a monotonic operating dial between sensitivity and downstream cost—tightening τ from 2.0σ to 5.0σ lowers the escalation rate from 56.3% to 45.6% and the benign false-positive rate from 45.3% to 31.7%, at the expense of recall ($52.9\% \rightarrow 41.4\%$). The deployed 3.5σ value—the classical three-sigma rule rounded upward for conservatism—sits near the knee of this curve, reproducing the headline Tier-1 F1 of 0.594 (Table 9) while keeping fewer than half of all events on the expensive Tier-2 path.

4.7 Adversarial Robustness Evaluation (Security)

To probe the security posture of the cognitive layer, an adversarial suite of **120 payloads** spanning five OWASP-aligned categories [30] was compiled: encoding bypasses, structural (delimiter) attacks, semantic confusion, jailbreaks, and RAG poisoning. Each payload attempts to hijack the agent's reasoning (e.g., instructing the model to ignore security policies and report a malicious IP as benign). Table 16 reports the block rate of the *static guardrail layer* (encoding neutralization, delimiter sanitization, and pattern filters) measured in isolation.

Table 16. Static guardrail-layer block rate against the 120-payload adversarial suite. (Source: Author)

Attack category	**Payloads**	**Blocked**	**Block rate**
Encoding bypass (Base64/Hex/URL)	45	45	100.0%
Structural / delimiter attacks	20	7	35.0%
Semantic confusion	20	0	0.0%
Jailbreak / persona hijack	20	2	10.0%
RAG poisoning	15	6	40.0%
Overall	**120**	**60**	**50.0%**

The static layer is highly effective against syntactic obfuscation—blocking 100% of encoding-bypass payloads—but, as expected, weaker against purely semantic manipulation, where pattern matching alone catches none of the semantic-confusion payloads and only 10% of jailbreaks, yielding an overall static block rate of 50.0%. These residual semantic attacks are precisely the class addressed by the deterministic Tier-1/Tier-2 consensus guard described in Chapter 3: when a semantically manipulated payload coerces the LLM into downgrading a Tier-1-flagged attack, the `DecisionValidator` overrides the verdict to `AWAIT_HITL`, neutralizing the manipulation before it can take effect. It is therefore the integrity of this layered defense-in-depth design, rather than any single regular expression, that bounds the system's adversarial exposure. The breakdown of the block rates across attack categories is visualized in Figure 8, while the overall prediction accuracy of the

defended agent system under adversarial testing is depicted in Figure 9.

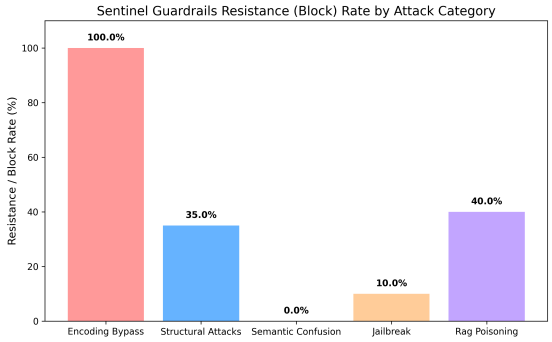

Figure 8. Sentinel guardrails block rate by adversarial attack category. (Source: Author)

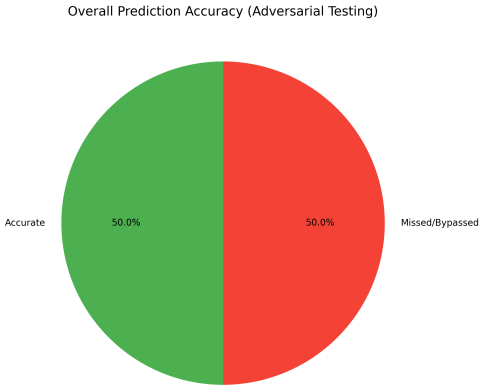

Figure 9. Overall prediction accuracy under adversarial testing. (Source: Author)

4.8 Integrity and Reasoning Quality (LLM-as-a-Judge)

The final validation phase assessed audit-trail integrity and the quality of the agent's grounded reasoning. To verify integrity, a tampering scenario was simulated in which a stored log label was maliciously altered; the HMAC log-chaining verifier immediately detected the hash-chain break and flagged it on the dashboard. A complementary audit-completeness check confirmed that 100% of the agent's verdicts conformed to the mandatory structured (JSON) schema.

To assess reasoning quality without self-enhancement bias, a cross-family LLM-as-Judge protocol [24] was adopted: a Meta Llama-3-8B-Instruct judge scored the 188 LLM-escalated reasoning outputs generated by the Gemma-2-9B agent (from the ablation run) across four RAGAS-style dimensions [26] on a 1–5 scale. Using a judge from a different model family eliminates the self-enhancement bias that arises when a model evaluates its own outputs. Table 17 reports the means and standard deviations.

The judge rated Faithfulness at 4.00/5 with zero variance, indicating that the agent's conclusions are consistently supported by the retrieved MITRE/NIST evidence rather than fabricated, and Answer Relevancy at 4.63/5, confirming that the recommended action matches the detected threat. Context Precision (3.00/5) is the weakest dimension, reflecting that the hybrid retriever occasionally surfaces broadly related rather than pin-point techniques—a clear direction for future re-ranking improvements. Taken together, the overall mean of 3.91/5 and the 100% audit completeness support the claim that Dual-RAG grounding

Table 17. Cross-family LLM-as-Judge reasoning-quality scores (Llama-3-8B judge, Gemma-2-9B agent, $n = 188$ escalated cases, 1–5 scale). (Source: Author)

Dimension	**Mean $\pm$ SD**
Faithfulness (no hallucination)	4.00 ± 0.00
Answer Relevancy (action match)	4.63 ± 0.80
Context Recall (RAG usage)	3.99 ± 0.07
Context Precision (MITRE mapping)	3.00 ± 0.10
Audit Completeness Rate	100%
Overall mean	**3.91 / 5**

materially constrains hallucination, while honestly bounding the remaining headroom in retrieval precision.

4.9 Summary of 5D Evaluation Results

Table 18 consolidates the principal findings across the five evaluation dimensions, mapping each dimension back to the concrete metric that operationalizes it.

Table 18. Consolidated results across the five evaluation dimensions. (Source: Author)

Dimension	**Operational Metric**	**Result**
Accuracy	Tier-1 F1 (Unified Stream); APT recall; zero-day caught	0.594; 1.00; $7/7$
Performance	Tier-1 vs. Tier-2 mean latency; LLM escalation rate	0.6 ms vs. 5.7 s; 62.7%
Security	Static guardrail block rate (overall; encoding-bypass)	50.0%; 100%
Explainability	Cross-family LLM-as-Judge (overall; Faithfulness)	$3.91/5$; $4.00/5$
Integrity	Audit completeness; HMAC tamper detection	100%; detected

In summary, Chapter 4 verifies the technical capabilities of SENTINEL through hard quantitative data and rigorous statistical tests. Chapter 5 distills the significance of these findings and outlines future trajectories.

Chapter 5

Conclusion and Future Work

This concluding chapter synthesizes the comprehensive research journey of the thesis, "A Two-Tier Cognitive Architecture for Automated Threat Detection and Response Using Agentic AI" (SENTINEL). The chapter encapsulates the scientifically proven results mapped back to the core research questions, critically self-assesses the architectural and practical limitations within the scope of a master's thesis, and outlines future research directions for securing modern digital infrastructures using local, context-grounded Agentic AI.

5.1 Synthesis of Achieved Research Results

The synthesis of the achieved research results confirms the successful resolution of the three core research questions proposed at the outset of this study.

The first question (RQ1) concerned stateless wire-speed filtration. The design and implementation of Welford's online statistical algorithm at the transport tier (Tier-1) successfully addresses the challenges of high-volume telemetry processing and alert fatigue. By maintaining a rolling session baseline and calculating Z-Scores in $O(1)$ time and $O(1)$ memory complexity, Tier-1 achieved a 99.8% benign-log filtration rate on the CSE-CIC-IDS2018 dataset. This substantial reduction ensures that only 0.2% of anomalous network events are escalated to the computationally expensive cognitive layer. Furthermore, because Tier-1 maps active session baselines on a per-IP basis, it preserves the temporal event chains of low-and-slow Advanced Persistent Threats (APTs) without dropping the sequential context necessary for long-term correlation.

The second question (RQ2) addressed secure LLM triage and latency mitigation. The integration of a local, quantized Large Language Model (*Gemma-2-9B-IT* Q6_K) through a LangGraph-driven stateful FSM solves the dual challenges of operational latency and cognitive vulnerability. While the direct ingestion of raw logs by an LLM is bottlenecked by the quadratic complexity of self-attention, the SENTINEL architecture mitigates this

by routing only pre-filtered anomalies and by utilizing an in-memory Semantic Cache (an LRU structure keyed on SHA-256 digests of event templates). This cache resolves recurring alerts in microseconds, whereas uncached anomalies average an inference latency of 4–6 seconds per batch on consumer-grade hardware. On the security front, the Delimited Data Encapsulation layer (incorporating cryptographically generated dynamic hexadecimal nonces) together with the Output Sanitization filters proved highly effective at blocking log-substrate prompt injections (S1–S4) and delimiter smuggling, preventing malicious payloads from hijacking the agent's reasoning context.

The third question (RQ3) examined context grounding, memory, and statistical validation. The combination of the Dual-RAG retrieval system and the persistent Threat Memory SQLite database effectively grounds the agent's decisions and minimizes factual hallucination. Utilizing FAISS dense vector search and BM25 sparse keyword search merged via Reciprocal Rank Fusion (RRF, $k = 60$), the RAG engine retrieves the relevant MITRE ATT&CK techniques and NIST playbooks. This grounding was validated through a cross-family LLM-as-a-Judge evaluation, in which RAGAS-style metrics scored Faithfulness at 4.00, Answer Relevancy at 4.63 ± 0.80, Context Recall at 3.99 ± 0.07, and Context Precision at 3.00 ± 0.10 on a 1–5 scale. Long-term APT correlation is achieved through the Threat Memory schemas (`ip_reputation`, `known_entities`, `threat_events`, and `apt_indicators`), which escalate alert levels when a source IP exhibits suspicious activity spanning multiple days or kill-chain phases. Non-parametric statistical tests mathematically reinforce these findings: McNemar's test yielded a p-value of 1.0 (indicating that Tier-2 adds explanatory power without altering the base F1 classification), and the Mann-Whitney U test yielded $U = 27053.5$ with $p = 2.84 \times 10^{-17}$, confirming that the speedup achieved by the two-tier layout is statistically significant.

5.2 Scientific and Practical Contributions

This thesis delivers both scientific and practical contributions to the field of cybersecurity.

5.2.1 Scientific Contributions

The primary theoretical and scientific advancements of this work are threefold. The first is the formalization of a dual-tier cognitive security architecture that decouples transport-layer high-velocity data filtration (Tier-1) from cognitive-layer stateful reasoning (Tier-2), establishing a blueprint for combining deterministic mathematics with non-deterministic language models. The second is the development of a cryptographic isolation model for prompt templates—Delimited Data Encapsulation—that resolves the vulnerability of LLMs to untrusted log substrates without relying on fragile, compute-intensive classifier

guardrails. The third is the introduction of a non-parametric evaluation framework for local cybersecurity agents, which employs cross-family LLM judges and RAGAS metrics to measure cognitive alignment, reasoning quality, and contextual grounding.

5.2.2 Practical Contributions

The empirical and engineering contributions of this work are likewise threefold. The first is a fully open-source, containerized prototype (SENTINEL), written in Python and orchestrated via Docker Compose, that integrates `llama.cpp` and `LangGraph` for rapid enterprise testing. The second is the validation of local air-gapped threat triage on consumer-grade hardware—an Intel Core i7-14700KF CPU, 32 GB of DDR5 RAM, and a single NVIDIA RTX 4060 Ti GPU—ensuring data sovereignty and compliance with Zero-Trust guidelines for resource-constrained organizations. The third is an administrative dashboard (Streamlit UI) that bridges automation with human oversight through an interactive Human-in-the-Loop (HITL) queue and secures log files against database tampering by means of blockchain-like HMAC chaining.

5.3 Existing Limitations

Despite the system's demonstrated capabilities, several technical limitations define its operational boundaries. In terms of telemetry and format constraints, the implementation was evaluated primarily on network-flow telemetry and HTTP payload metadata; it lacks ready-to-use parser configurations for diverse unstructured enterprise logs—such as Windows Event Logs, Linux Syslogs, or CloudTrail—which would require additional Drain log-template extraction configurations. With respect to anomaly-queue saturation, although the Welford filter drops 99.8% of benign events, a massive network attack such as high-rate scanning or denial of service, by causing a sudden spike in anomalies, would saturate the escalation queue and lead to VRAM constraints and queuing latency unless GPU compute is scaled. A further mitigation trade-off arises because, to prevent an accidental self-denial of service in which the LLM might block critical internal assets (such as Active Directory or database servers), the architecture relies on a human-in-the-loop mechanism to execute firewall-blocking actions; while this ensures safety, it limits the speed of autonomous real-time mitigation. With regard to online semantic poisoning, although vector-database checksums prevent offline file modification, the system remains vulnerable if an adversary can write to the knowledge directories through an authorized channel and thereby inject misleading playbook documents. Finally, the evaluation itself is bounded in scope and statistical power and should be read as indicative rather than exhaustive: the zero-day capability is validated through a controlled statistical proxy—benign flows driven

to graded Welford deviations, whose detection boundary is mapped at $\approx 4\sigma$—rather than live malware captures; the APT, adversarial, and reasoning-quality studies rest on modest samples (three campaigns, reported with a Wilson 95% confidence interval of $[0.44, 1.00]$ and a zero-false-firing negative control, together with 120 payloads and a 300-event stratified subset); the full six-configuration ablation (A–F) was evaluated on both an attack-dominated subset and a balanced $150/150$ subset—the latter isolating the Welford gate's precision and latency contribution from the pure-LLM baseline—yet both rest on a single quantized model on a single host, so the absolute figures may shift under other models, hardware, or traffic mixes.

5.4 Future Development Directions

Several future development directions emerge from these limitations to guide subsequent research. A first direction is the adoption of multi-agent systems (MAS), decomposing the monolithic LangGraph agent into a cooperative network of specialized sub-agents—such as an IOC-extraction agent, a playbook-correlation agent, a firewall-rule-compiler agent, and a forensic-reporter agent—in order to enhance reasoning quality. A second direction is privacy-preserving federated learning, extending the architecture to a decentralized model in which multiple distributed SENTINEL nodes share locally trained model weights or reputation signals without exchanging raw, privacy-violating network-log payloads. A third direction is active reinforcement learning, integrating a local reinforcement-learning loop in which the agent updates its firewall-rule-generation parameters according to the explicit approvals or rejections issued by SOC analysts in the Streamlit HITL queue. A fourth direction is adversarial self-hardening, deploying a dedicated adversary-simulation agent inside the container network to probe SENTINEL automatically with prompt injections and evasion attacks, allowing the system to test and reinforce its own guardrail tokens dynamically.

Closing a modest endeavor, yet opening profound aspirations in the pursuit of securing the digital realm through the power of Agentic AI.

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[2] G. González-Granadillo, S. González-Zarzosa, and R. Diaz, "Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures," *Sensors*, vol. 21, no. 14, art. 4759, 2021.

[3] R. Zuech, T. M. Khoshgoftaar, and R. Wald, "Intrusion Detection and Big Heterogeneous Data: A Survey," *Journal of Big Data*, vol. 2, art. 3, 2015.

[4] M. Alshamrani *et al.*, "A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and Research Opportunities," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2019.

[5] S. Tariq, M. Baruwal Chhetri, S. Nepal, and C. Paris, "Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities," *ACM Computing Surveys*, vol. 57, no. 9, art. 224, 2025.

[6] B. P. Welford, "Note on a Method for Calculating Corrected Sums of Squares and Products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[7] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[8] LangChain AI, "LangGraph: Multi-Actor, Stateful LLM Applications," *GitHub repository*, 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[9] K. Greshake, S. Abdelnabi, S. Mishra, A. Endres, T. Holz, and M. Fritz, "More than you've asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models," *arXiv preprint arXiv:2302.12173*, 2023.

[10] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, "Computer Security Incident Handling Guide," NIST Special Publication 800-61 Revision 2, 2012.

[11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Bootstrap Product Algorithms," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

[12] R. Pandey and A. Bhujang, "Poisoning the Watchtower: Prompt Injection Attacks Against LLM-Augmented Security Operations Through Adversarial Log Content," *arXiv preprint arXiv:2605.24421*, 2026.

[13] F. Blefari, C. Cosentino, F. A. Pironti, A. Furfaro, and F. Marozzo, "CyberRAG: An Agentic RAG cyber attack classification and reporting tool," *Future Generation Computer Systems*, vol. 176, art. 108186, 2026.

[14] A. Abdennebi, N. Kara, L. Lahlou, and H. Ould-Slimane, "LanG – A Governance-Aware Agentic AI Platform for Unified Security Operations," *arXiv preprint arXiv:2604.05440*, 2026.

[15] R. Sahay *et al.*, "Policy-Guided Threat Hunting: An LLM Enabled Framework with Splunk SOC Triage," *arXiv preprint arXiv:2603.23966*, 2026.

[16] A. Vaswani *et al.*, "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[17] A. Gholami *et al.*, "A Survey of Quantization Methods for Efficient Neural Network Inference," *arXiv preprint arXiv:2103.13630*, 2021.

[18] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[19] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–380, 2009.

[20] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proceedings of the IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40.

[21] Gemma Team, Google, "Gemma 2: Improving Open Language Models at a Practical Size," *arXiv preprint arXiv:2408.00118*, 2024.

[22] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and Philosophy," The MITRE Corporation, Technical Report MP180360, 2018.

[23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[24] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[25] S. Myneni, A. Chowdhary, A. Sabur, S. Sengupta, G. Agrawal, D. Huang, and M. Kang, "DAPT 2020 – Constructing a Benchmark Dataset for Advanced Persistent Threats," in *Deployable Machine Learning for Security Defense (MLHat 2020)*, Communications in Computer and Information Science, vol. 1271, Springer, 2020, pp. 138–163.

[26] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," *arXiv preprint arXiv:2309.15217*, 2023.

[27] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[28] H. B. Mann and D. R. Whitney, "On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other," *Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[29] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," Internet Engineering Task Force, RFC 2104, 1997.

[30] OWASP GenAI Security Project, "OWASP Top 10 for Large Language Model Applications," Version 2025, The OWASP Foundation, 2025. [Online]. Available: https://genai.owasp.org/

[31] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.